DTU Compute
Department of Applied Mathematics and Computer Science

Reading the Reader

Adaptive Reading Systems: A Modular Software Architecture

Master Thesis

Reading the Reader
Adaptive Reading Systems: A Modular Software Architecture

Master Thesis
Date, year

By
Sachin Baral
Satish Gurung

Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Bygning 324„ 2800 Kgs. Lyngby Denmark
https://www.compute.dtu.dk/
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Acknowledgements

This thesis was carried out at DTU Compute, Department of Applied Mathematics and Computer Science, as part of the Reading the Reader research programme, in partial fulfilment of the requirements for the MSc in Computer Science and Engineering at the Technical University of Denmark.

We would like to thank our main supervisor, Per Bækgaard (Associate Professor, DTU Compute), for proposing this project and for his guidance and feedback throughout its course. We are equally grateful to our co-supervisors, Ashkan Tashk and Chaudhary Muhammad Aqdus Ilyas (Postdocs, Cognitive Systems, Department of Applied Mathematics and Computer Science, DTU), for their advice and continued support during the work.

We also thank the wider Reading the Reader programme at DTU Compute, whose prior theses and ongoing research provided the foundation on which this work builds.

Finally, we thank our families and friends for their support and patience throughout this work.

Sachin Baral
Satish Gurung
Kongens Lyngby, Date, year

Contents

1 Introduction

1.1 Motivation and Context

Reading is a foundational skill that underpins participation in education, employment, and daily life. For many people, however, reading is not a straightforward activity. The World Health Organization estimates that at least 2.2 billion people worldwide live with a vision impairment [1], and age-related macular degeneration (AMD) alone is projected to affect 288 million people globally by 2040 [2]. Beyond vision impairment, reading difficulties arise from a wide range of cognitive and perceptual factors, affecting learners at every stage of life. In each of these cases the act of reading places demands that the reader's visual or cognitive system can only partially meet, and the gap between what the text requires and what the reader can comfortably process translates directly into slower reading, greater fatigue, and reduced comprehension.

Digital text offers something that printed text cannot: the presentation can be changed in real time. Font size, letter spacing, line height, contrast, and weight are all adjustable on a screen, and adjustments can be made not on a fixed schedule but in response to how the reader is actually engaging with the content at any given moment. Eye tracking provides exactly the kind of signal needed to drive such responses. A gaze-sensing device placed in front of a reader produces a continuous stream of data that reflects where the reader is looking, how long they dwell on particular regions, and when their reading slows or falters. A system that can interpret this stream and translate it into small, targeted typographic adjustments has the potential to provide individualised reading support without interrupting the reader's flow. This is the core premise of adaptive reading technology.

This promise, however, carries an inherent risk. Fig. 1.1 shows the kind of typographic adaptation such a system applies, increasing size, spacing, and contrast to make a line easier to read. An adaptation like this can support a struggling reader, but the same change applied at the wrong moment, or too abruptly, can cost the reader their place and force them to start the line again. The central design challenge of adaptive reading is therefore not simply deciding what to change, but changing it in a way that supports the reader without disrupting the very reading flow the system is trying to protect. This tension between adaptation and continuity runs through every layer of the platform this thesis develops.

The Reading the Reader project at DTU Compute, funded by the Novo Nordisk Foundation with approximately DKK 8 million, brings together computer scientists, typographers, psychologists, and ophthalmologists to develop exactly such a system [3, 4]. The programme targets readers across a range of difficulties (including central vision loss caused by AMD and reading disabilities such as dyslexia) with the shared premise that eye-tracking data can reveal individual reading patterns and inform personalised typographic adaptation. Prior work within the programme has demonstrated that real-time, gaze-driven adaptive reading is technically achievable, but the resulting prototypes were tightly coupled systems in which sensing, decision logic, and the reading interface were woven into a single codebase [5, 6, 7]. The cost of this coupling is concrete: each new study tended to extend the system by copying the previous prototype's frontend wholesale rather than by building on a shared platform.

Consider a researcher who wants to test a simple hypothesis: that fading a font change gradually into place is less disruptive to the reader than applying it abruptly. On the exist-

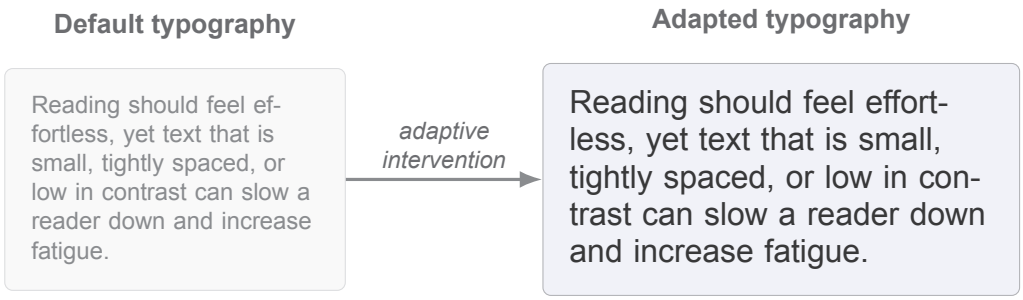


Figure 1.1: A typographic intervention of the kind an adaptive reading system applies, shown as a before-and-after comparison of the same sentence. The adapted version on the right increases font size, line spacing, and contrast to improve legibility. Changes such as this can support a reader, but if applied abruptly or at the wrong moment they risk disrupting reading flow; reconciling these two effects is the central challenge of this thesis.

ing prototypes, answering this requires editing core application code, and running the two conditions under identical sensing means maintaining two divergent forks of the system. The question is scientifically straightforward, but the architecture makes it expensive to ask. A platform on which such a comparison is a configuration choice rather than a code change would remove that friction, and providing it is precisely the gap this thesis addresses: the modular infrastructure the programme needs to move from proof-of-concept prototypes to rigorous, repeatable experimentation.

1.2 Goal of the Project

The goal of this project is to design and implement a modular, researcher-operated adaptive reading platform in which sensing, eye-movement analysis, decision strategy, and intervention execution are independently replaceable components behind stable contracts. The aim is not to measure a specific typographic effect or to train a reading model; it is to build the infrastructure that makes such measurements and models possible in a principled and reproducible way, and to demonstrate that the architecture meets the properties the programme requires.

Building a software platform might appear to be an engineering exercise rather than a research one. We argue that it is both. The research contribution of this thesis is not the running code itself but the design knowledge embodied in it: the module boundaries and integration contracts that determine whether sensing, analysis, decision, and intervention can genuinely be separated, and the evidence, obtained by evaluating the implemented system against explicit criteria, of whether those boundaries hold under the demands of a real-time reading loop. This framing follows the Design Science Research paradigm, in which a designed artifact and the knowledge it embodies are themselves the contribution, provided they are rigorously motivated and evaluated [8]. The methodology that operationalises this stance is developed in chapter 3.

To reach this goal, the project pursues four objectives.

- **Modular architecture.** To design an architecture that separates sensing, eye-movement analysis, decision strategy, and intervention execution into independently replaceable components behind stable contracts, so that any layer can be extended or substituted without modifying the core reading runtime.
- **Real-time sensing pipeline.** To implement an end-to-end pipeline that integrates

physical Tobii hardware and converts the raw gaze stream into reading-behaviour signals online, within a latency budget compatible with live adaptation.

- **Researcher experiment workflow.** To build researcher-facing tools for operating controlled reading experiments, including live observation of the participant's view, manual intervention control, and complete session export and replay.
- **External decision-provider contract.** To define an integration boundary through which AI-driven or alternative decision strategies can be connected to the running platform without modifying its core.

The work is scoped to thesis-scale controlled experiments with general laboratory participants. It does not implement a machine learning model for reading state classification, does not target PDF as a reading format, and does not pursue clinical recruitment of vision-impaired or dyslexic participants. The platform is designed to support those ambitions in future work; validating them is outside the scope of this thesis.

The research questions that govern the evaluation of the resulting platform are stated in full at the end of chapter 2, once the supporting literature has established their motivation (Sec. 2.5).

1.3 Thesis Outline

The remainder of this thesis is organised as follows.

chapter 2 surveys the concepts and literature that underpin the work. It defines the key terms used throughout the thesis, reviews related work on eye tracking in reading, adaptive reading systems, and the existing market landscape, and identifies the gaps that this thesis addresses. The chapter closes with the refined problem statement and four research questions that govern the rest of the work.

chapter 3 describes how the thesis was conducted. It positions the work within the Design Science Research paradigm, explains how requirements were elicited, describes the iterative development process, sets out the evaluation strategy, and addresses ethical considerations arising from the use of eye-tracking data.

chapter 4 derives the functional and non-functional requirements of the platform from stakeholder analysis and use cases. It introduces a domain model and documents the constraints that bound the design space.

chapter 5 presents the system architecture. It introduces the four-layer modular design, describes the module contracts and data flow, and records the key design decisions and their rationale.

chapter 6 describes how the architecture is realised in code. It covers the monorepo structure, selected implementation highlights across the sensing, decision, and intervention layers, and the build and CI workflow.

chapter 7 evaluates the platform against the four research questions. It reports on capability demonstration, calibration validation, sensing pipeline latency, and session data export analysis, and assesses the degree to which the platform meets each criterion.

chapter 8 interprets the evaluation results in relation to the problem statement and the literature, discusses the limitations of the current work, and outlines directions for future research.

chapter 9 summarises the contributions and answers the research questions directly. It introduces no new content.

2 Key Concepts and Related Work

This chapter establishes the conceptual and scholarly foundation for the thesis. Sec. 2.1 defines the domain terms that the rest of the thesis builds on; Sec. 2.2 surveys the academic work most directly relevant to adaptive reading and gaze-based intervention; Sec. 2.3 surveys the current tool and product landscape; Sec. 2.4 identifies the gaps that motivate the thesis contribution; Sec. 2.5 states the refined problem and measurable research questions; and Sec. 2.6 summarises the chapter.

2.1 Key Concepts for Adaptive Reading Analytics

This section defines the core concepts underlying the thesis. Definitions are grounded in primary and project-aligned sources.

2.1.1 Adaptive Reading System

The ”Reading the Reader” concept assumes a closed-loop system that senses gaze-related signals (including pupil size) during reading, extracts higher-level events (fixations, saccades, regressions), classifies reading state or style, and triggers typographic interventions in real time [3]. Fig. 2.1 depicts this loop as four sequential stages, sensing, analysis, decision, and intervention, whose output changes what the reader sees and so feeds the next cycle of sensing. Treating each stage as an independently replaceable module behind a stable contract is the central architectural commitment of this thesis, and it is the property examined in the chapters that follow.

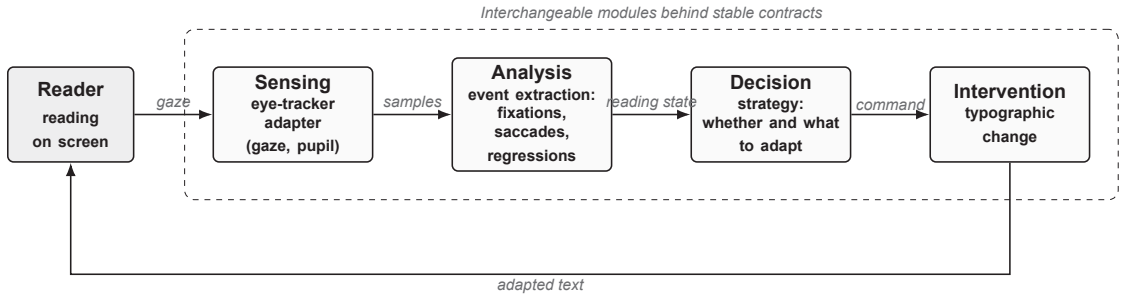


Figure 2.1: The closed-loop adaptive reading concept. Gaze and pupil signals from the reader are sensed, reduced to oculomotor events, classified into a reading state, and used to decide and apply a typographic intervention, which changes what the reader sees and thereby closes the loop. The four stages are designed as interchangeable modules behind stable contracts.

2.1.2 Micro-Interventions and Just-in-Time Adaptation

Micro-interventions are an established pattern in digital-health software rather than a term coined here. Persson et al. synthesise this literature into the Design for Microintervention Software Technology (D-MIST) framework, which organises a micro-intervention system into three levels: the narrative, or the consolidated experience of engaging with many micro-interventions toward an overarching goal; the micro-intervention itself, a focused intervention composed of self-contained events; and the event, a single momentary change delivered through a resource [9]. The Reading the Reader proposal adapts this concept to reading, framing micro-interventions as small, context-preserving changes to typography or layout (for example, spacing or contrast adjustments) that support reading without disrupting flow [3].

In this thesis, just-in-time adaptation is the term we use for an intervention policy that fires on short temporal windows of online gaze and interaction features rather than on a fixed schedule. We define it operationally as a policy that triggers within such a window while controlling for two quantities: the intervention cost, meaning the risk of disrupting reading, and the recovery behaviour, meaning the time the reader needs to resume reading afterwards.

2.1.3 Context Preservation and Recovery Metrics

Recent work presented at the Symposium on Eye Tracking Research and Applications (ETRA) frames context preservation as interface functionality that enables readers to resume reading quickly after a typographic change. It introduces the metric Reading Resume Time (RRT) and compares four intervention designs (Popup, Undo, Notification, and Gradual) on the time a reader takes to recover after the change [10].

Jensen et al. report that, among the four designs, only the gradual intervention combined with highlighting produced a statistically significant reduction in intervention time, falling from 14.9 s without highlighting to 4.0 s with it (Mann-Whitney U, $p < .001$), while the gradual design was also associated with the lowest reported recovery difficulty [10]. This indicates that intervention form factor and transition smoothness materially affect how quickly reading recovers.

This thesis treats context preservation as a first-class design goal whose effect should be observable rather than assumed. We therefore propose to instrument the runtime so that candidate recovery measures, such as Reading Resume Time, the time taken for gaze to restabilise after a change, and any burst of regressions that follows it, can be logged and compared in future studies, rather than claiming a measured recovery result for any single intervention in this work.

2.1.4 Central Vision Loss and Reading

The wider Reading the Reader project aims to improve reading accessibility for people with impaired vision or cognition [3, 4], with central vision loss (most commonly caused by age-related macular degeneration, AMD) representing the primary motivating condition [5]. Central vision loss (CVL) damages the part of the visual field used for high-acuity tasks, forcing affected readers to rely on peripheral or parafoveal vision and producing measurable declines in reading speed and changes in eye-movement patterns [11]. Reading is therefore not a single perceptual act but a combination of sensory, motor, and cognitive processes that CVL disrupts simultaneously.

Interventions that adapt presentation to the individual reader have been studied as a way to recover some of this lost performance. Perceptual-learning regimes have been reported to improve reading speed in people with long-standing central vision loss [12], and a broader literature surveys clinical and technological strategies for training reading skills in central-field-loss patients [13]. Mobile and digital assistive technologies have been positioned as a discreet, widely available route to supporting visually impaired readers in everyday settings [14]. A distinction of scope is important here. Central vision loss is the motivating target condition for the wider Reading the Reader project, but the reading sessions in this thesis are conducted with general laboratory participants rather than recruited CVL patients. The platform is therefore designed to serve CVL readers eventually, while the present work validates the architecture and runtime rather than a clinical outcome. This thesis does not pursue a clinical training protocol; it treats CVL as the condition that justifies real-time, individualised typographic adaptation and that frames why minimising disruption (Sec. 2.1.3) matters.

2.1.5 Oculomotor Events: Fixations, Saccades, and Regressions

Gaze during reading is not continuous but decomposes into discrete oculomotor events. A fixation is a relatively stable gaze on a small region, during which visual information is extracted; a saccade is the rapid ballistic movement that relocates the gaze between fixations [15]. A regression is a backward saccade against the reading direction and is commonly associated with processing or comprehension difficulty [16]. Fig. 2.2 illustrates these events two ways for a single line of text: panel (a) shows the spatial scan path over the words, while panel (b) replots the same gaze as position against time, where fixations appear as plateaus whose width is their dwell duration and saccades as the steep steps between them.

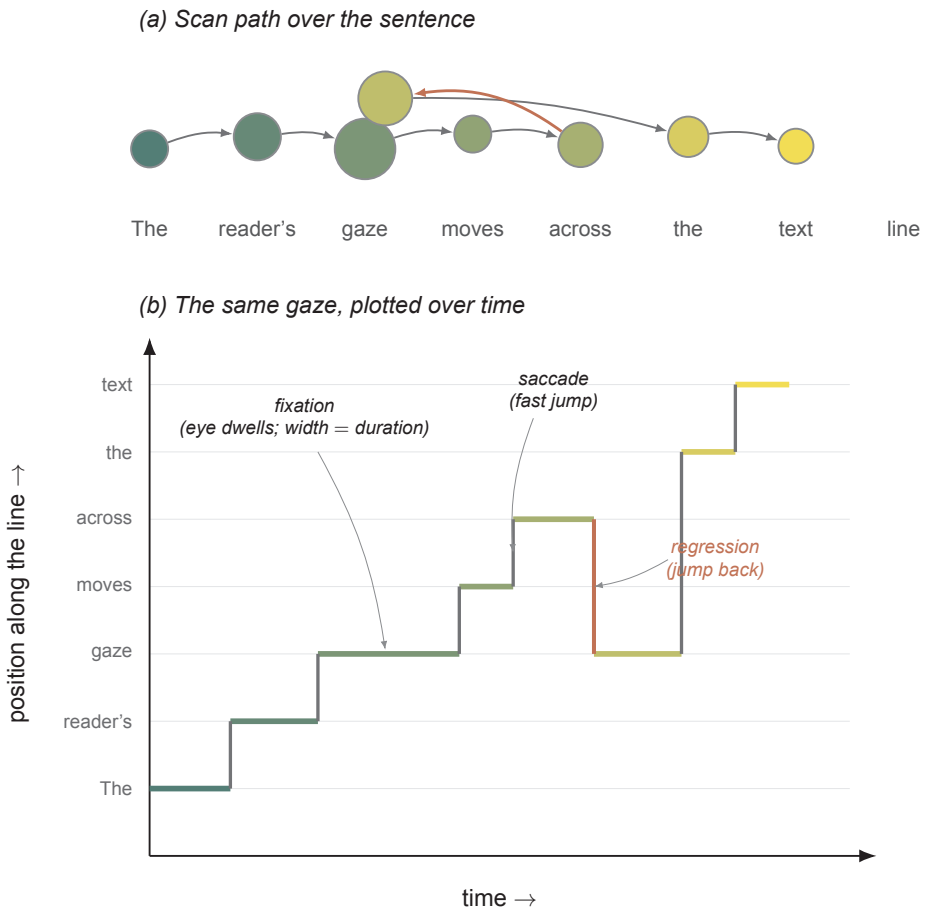


Figure 2.2: Two complementary views of a representative reading scan path over a single line of text. (a) The spatial scan path: each circle is a fixation positioned over the word it lands on, with circle area indicating dwell duration and fill colour indicating reading order; grey arrows are forward saccades and the red arrow is a regression to an earlier word. (b) The same gaze plotted as position along the line against time: fixations become horizontal plateaus whose width is their dwell duration, saccades become steep vertical steps, and the regression becomes a backward (downward) step. Fixations keep the same reading-order colour in both panels, so the two views correspond.

Extracting these events from a raw gaze stream is the task of fixation-identification algorithms, for which Salvucci and Goldberg proposed an influential taxonomy organised by the criterion each algorithm uses, such as velocity, dispersion, or area of interest [15]. The platform developed in this thesis performs this extraction online using the area-of-interest variant of that taxonomy: each gaze observation is first resolved to the word it

falls on, and the dwell time on that word is thresholded into fixations and saccades, so that downstream decision strategies can reason over reading behaviour rather than over raw coordinates. These definitions therefore underpin the sensing and analysis layers described in later chapters.

2.1.6 Typography as an Intervention Medium

The adaptations this thesis applies are typographic: changes to font, size, weight, spacing, and contrast. This choice is grounded in evidence that such variables measurably affect readability. Adjustable typography has been argued as a route to low-vision text accessibility [17], and controlled studies report that increased letter spacing and greater letter width improve reading acuity for low-vision readers [18], that font size and line spacing affect online readability [19], and that background colour influences reading performance [20]. Eye-tracking has itself been used to study how font size and type influence online reading [21].

Taken together, these findings define the parameter space within which micro-interventions (Sec. 2.1.2) operate. They justify treating typography as the actuator of the adaptive loop while leaving the decision of which adjustment to make, and when, to the interchangeable strategy modules that are the architectural focus of this thesis.

2.2 Related Academic Work

This section reviews the work most directly relevant to the thesis along three threads: the prior projects within the Reading the Reader initiative, the use of eye-tracking as a real-time signal in reading research, and the design and evaluation of typographic interventions. Rather than cataloguing each source in isolation, the discussion draws out where the findings converge, namely that adaptive reading is feasible and that intervention design measurably shapes the reading experience, and where they leave a gap that this thesis addresses.

2.2.1 Prior Work in the Reading the Reader Project

This thesis continues a line of student projects within the Reading the Reader initiative, and its contribution is best understood relative to that lineage. Refsgaard and Farooq built the first flexible reading application and gaze data-collection platform for the project, conducting an experiment with 23 participants to assess the feasibility of the interface and data pipeline [5]. Building directly on that codebase, Pereira and Andrei developed an adaptive e-reader that triggers typographic adjustments and evaluated four intervention designs (an Undo, a Popup, a Notification, and a Gradual change) through an eye-tracking study, reporting that careful typographic changes can improve reading speed and comfort while differing markedly in disruption and user acceptance [6]. In parallel, Palinec developed a reactive adaptive frontend intended to consume dynamic machine-learning output and adjust text presentation in real time [7].

These projects share a common limitation that motivates the present work. Each delivered a largely self-contained prototype, and the work evolved incrementally, with each project extending the previous frontend rather than building against a stable platform contract; Pereira and Andrei, for example, based their prototype on the earlier Refsgaard and Farooq code [5, 6]. This thesis responds by separating sensing, analysis, decision strategy, and intervention execution into independently replaceable modules, so that future intervention designs and decision providers can be integrated without rewriting the reading runtime or the researcher workflow.

2.2.2 Eye-Tracking in Reading Research

A body of work establishes that eye movements are a productive signal for understanding and enhancing reading. The eyeBook demonstrated an interactive reading application that detects the reader's position on the screen from gaze and generates contextual feedback, illustrating that gaze can drive in-situ responses during reading rather than only post-hoc analysis [22]. Subsequent work has shown that eye-movement behaviour can predict reading performance in online contexts [16] and that reading versus not-reading behaviour can be classified automatically from eye-movement features [23]. These results depend on first reducing raw gaze to oculomotor events using identification algorithms of the kind catalogued by Salvucci and Goldberg [15]. Collectively they support the architectural premise of this thesis: that a real-time analysis layer converting gaze into fixations, saccades, and regressions is a necessary foundation for any decision strategy that acts on reading behaviour.

2.2.3 Context Preservation and Intervention Design

The context-preservation paper formalises a key human-computer interaction (HCI) challenge: adaptive reading interfaces can either assist or hinder reading depending on how changes are introduced and how recovery is supported. In a within-subjects study, Jensen et al. compared intervention times against a non-highlight baseline from earlier project work [6] and found that only the gradual intervention improved significantly when paired with highlighting (Mann-Whitney U, $p < .001$), while the popup, undo, and notification interventions showed no significant difference, an outcome the authors attribute partly to the small sample size [10]. The design of the interventions themselves has also been examined within the Reading the Reader project: Pereira and Andrei compared four intervention forms (Undo, Popup, Notification, and Gradual) and reported that they differ markedly in disruption and user acceptance [6]. The typographic parameters these interventions manipulate are in turn grounded in controlled readability studies, which show that font size and line spacing [19] and font size and type [21] measurably affect online reading. Together these results support the thesis argument that intervention design is a measurable system behaviour, one that must be architecturally supported through stable integration hooks and logging mechanisms.

2.2.4 Transparent and Reproducible Analysis Pipelines

A theme common to the studies above is that the value of gaze analysis rests not only on which metrics are computed but on how reproducibly they are produced. Each result depends first on reducing raw gaze to oculomotor events through an explicit identification algorithm, for which Salvucci and Goldberg provide a widely used taxonomy organised by the criterion each applies, such as velocity (I-VT) or dispersion (I-DT) [15]. Only once this extraction step is fixed and documented can downstream results, such as predicting reading performance [16] or classifying reading versus non-reading behaviour [23], be meaningfully compared or replicated across studies.

This carries a direct architectural consequence for the thesis: the analysis layer must be an explicit, inspectable stage rather than logic buried inside the reading runtime, so that the events driving every intervention can be logged, audited, and replayed. A modular and transparent pipeline is therefore not only good experimental practice but a precondition for the systematic comparison of interventions and decision strategies that motivates this work.

2.3 Market Landscape of Tools and Products

Alongside the academic literature, a survey of the commercial and product landscape clarifies what adaptive reading technology is already available and where the gaps lie.

This section reviews representative products across three groups that bear on this thesis: research-grade eye trackers and experiment software, accessibility and reading-assessment aids, and built-in adaptive reading interfaces. The survey is not exhaustive; it samples widely used products in each group to characterise prevailing capabilities and delivery models rather than to rank vendors. Tab. 2.1 summarises the comparison, after which the implications for this thesis are discussed.

2.3.1 Comparison of Tools

Table 2.1: Representative tools and products in the adaptive reading and eye-tracking landscape, grouped by category and compared on delivery model, capabilities, and a pricing signal.

Category	Vendor / Product	Delivery Model	Capabilities	Pricing Signal
Screen-based eye tracker	Tobii Pro Fusion	Hardware	Up to 250 Hz sampling; fixation/saccade detection; pupil diameter recording [24]	Quote-based [24]
Screen-based eye tracker	EyeLink 1000 Plus	Hardware	Binocular sampling up to 2000 Hz; research integration support [25]	Quote request [26]
Lower-cost eye tracker	GazePoint GP3 HD	Hardware + Software	150 Hz sampling; research-grade positioning [27]	$2,650 hardware-only [27]
Wearable eye tracking	Pupil Labs Neon	Hardware	Academic pricing; complete research system [28]	€5,515 academic [28]
Remote webcam tracking	RealEye	SaaS	Subscription plans; data export; panel workflow [29]	Panelists $15 [30]
Experiment software	Tobii Pro Lab	Desktop software	Experiment design + recording + analysis [31]	Trial available [31]
Reading assessment	Lexplore	Service + Hardware	AI-based reading assessment [32]	Quote-based [33]
Accessibility device	OrCam Read 3	Hardware	Reads printed/digital text aloud [34]	$116/month option [34]
Adaptive interface	Microsoft Immersive Reader	Built-in feature	Line Focus; Text Spacing; read-aloud [35]	Included [35]
E-reading customisation	Amazon Kindle settings	Device feature	Font size and appearance adjustments [36]	Included with device [36]

Three observations follow from Tab. 2.1. First, the research-grade eye trackers and experiment software offer the sampling rates and gaze precision this thesis requires, but they are delivered as closed hardware and analysis suites priced by quotation or at several thousand euros [24, 25, 27, 28, 31], and they are oriented toward recording and post-hoc analysis rather than real-time, gaze-driven adaptation of the text being read. Second, the accessibility and reading-assessment tools act on the reading material, but either as static customisation under user control [34, 36] or as assessment services that analyse

the reader without adapting presentation in the loop [32]. Third, the built-in adaptive interfaces provide useful presentation controls such as line focus and text spacing [35], yet these are manually selected features rather than responses driven by the reader's gaze. No surveyed product combines real-time gaze sensing, automatic typographic adaptation, and researcher-facing experiment control within a single modular platform, which is precisely the combination this thesis targets. Fig. 2.3 arranges the surveyed products along two of these dimensions, whether the system adapts the text presentation and whether it uses the reader's gaze, making the region they leave unoccupied immediately visible.

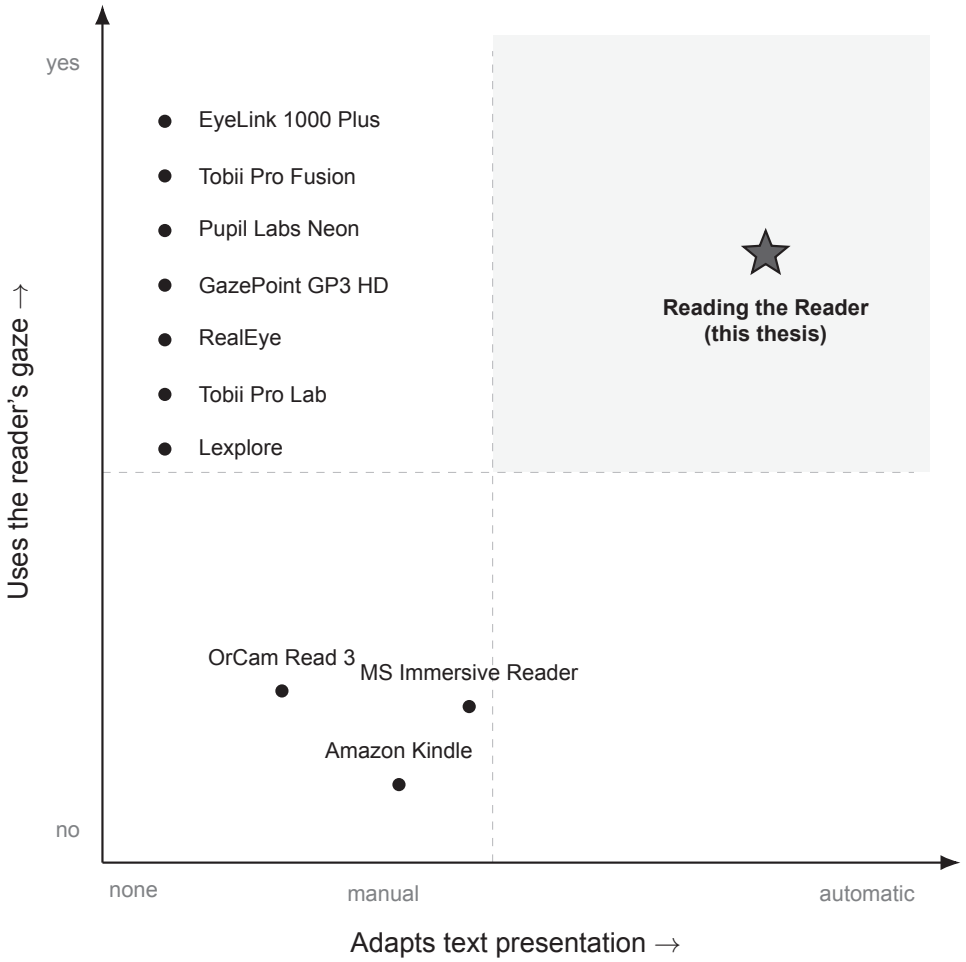


Figure 2.3: Positioning of the surveyed tools on two dimensions: whether the system adapts the text presentation (horizontal) and whether it uses the reader's gaze (vertical). The research-grade trackers and analysis software share the leftmost position because none of them adapt the text; they differ only in how directly they expose a usable gaze signal. The accessibility and built-in reading tools sit at the lower band: they change presentation, but manually and without gaze. The upper-right region, combining automatic gaze-driven adaptation with the researcher experiment control this thesis adds, is unoccupied. Positions reflect the authors' assessment of the sources compiled in Tab. 2.1.

2.4 Gaps and Opportunities

2.4.1 Architecture Gap: Experimentability

The prior Reading the Reader theses each delivered a working prototype, but each was tightly coupled and was extended by copying the previous frontend rather than by plugging into a shared platform [5, 6, 7]. This coupling hinders the systematic comparison

of interventions and decision strategies that the project ultimately requires. At the same time, empirical findings show that intervention design materially affects recovery and user preference [10], which makes such comparison scientifically important rather than merely convenient.

These coupling problems point to a clear opportunity. Designing the adaptation engine around hot-swappable intervention modules and auditable logging would enable systematic A/B testing, replay-based evaluation, real-time researcher override, and the transparent experimental comparison of interventions that the prior prototypes could not support.

2.4.2 Sensing Gap: Research-Grade versus Accessible Hardware

The market survey in Sec. 2.3 exposes a split in the sensing layer. Research-grade eye trackers deliver the sampling rate and precision needed for reliable fixation and saccade extraction, but they are closed and costly [24, 25, 27, 28], whereas the more accessible reading and accessibility tools do not expose a gaze signal that can drive adaptation [34, 35, 36]. A platform tied to one tracker therefore inherits both its cost and its constraints.

The opportunity is to place sensing behind a stable adapter interface so that the eye tracker is one interchangeable implementation among several. This decouples the reading runtime from any single vendor, allows research-grade hardware to be used where available while leaving room for lower-cost or webcam-based sensing, and keeps the door open to the multimodal signals (including pupil size) envisaged for the project [3].

2.4.3 Integration Gap: External and AI-Driven Decision Modules

None of the surveyed products expose a pluggable decision layer; each bundles fixed logic for when and how to act on the reader. This is at odds with the project's vision, in which decisions about adaptation may eventually be made by external, AI-driven providers rather than hard-coded rules [3]. Without a defined integration boundary, every new decision strategy would require modifying the core application.

The opportunity is to define a decision-strategy contract that the core application depends on, so that built-in rules and external or AI-based providers are interchangeable behind the same interface. This makes AI support an architectural property of the platform rather than a feature implemented end-to-end in this thesis, and it lets future contributors supply decision providers without touching the sensing, analysis, or intervention layers.

2.5 Final Problem Statement

The preceding review shows that adaptive reading is an active research area with a clear unmet need. Prior work in the Reading the Reader project produced capable but tightly coupled prototypes [5, 6, 7], the literature establishes that intervention design measurably affects reading recovery and preference [10], and the market survey confirms that no existing product unites real-time gaze sensing, automatic typographic adaptation, and researcher-facing experiment control in a single modular platform. The gap analysis in Sec. 2.4 reduced this to three concerns: experimentability, the coupling of the sensing layer, and the integration of external decision modules.

This thesis therefore addresses the following problem.

> How can a researcher-operated adaptive reading platform be architected so that sensing, eye-movement analysis, decision strategy, and intervention execution are independently replaceable, while supporting real Tobii-backed reading sessions and applying context-preserving micro-interventions without disrupting the participant's reading flow?

We decompose this into four research questions, each of which maps to evaluation criteria developed in chapter 7.

RQ1 (Modularity). Can the platform separate sensing, analysis, decision, and intervention into modules with stable contracts, such that a new intervention or decision provider can be added without modifying the core reading runtime?

RQ2 (Sensing pipeline). Can a real Tobii eye-tracking stream be converted into oculomotor events (fixations, saccades, and regressions) within a latency budget that preserves live adaptation during a reading session, in a manner that is inspectable and reproducible? The associated latency and feasibility criteria are defined and measured in chapter 7.

RQ3 (Context-preserving intervention). Can typographic micro-interventions be committed at controlled boundaries so that they adapt the text while preserving the reader's context, consistent with the recovery concerns identified in the literature [10]?

RQ4 (Researcher control and experimentability). Does the platform give the researcher the control and the auditable record (including session replay) needed to operate experiments and compare interventions and decision strategies?

These questions deliberately emphasise architecture and runtime feasibility over a controlled human-subjects effect study, in line with the thesis scope: the contribution is a defendable, modular platform on which interventions and external, AI-driven decision providers can be compared in future work, rather than an end-to-end empirical result for any single intervention.

2.6 Chapter Summary

This chapter established the conceptual and scholarly foundation for the thesis. It defined the core concepts of adaptive reading: the closed sensing-to-intervention loop, micro-interventions and just-in-time adaptation, context preservation with recovery metrics such as Reading Resume Time, the condition of central vision loss that motivates the work, the oculomotor events (fixations, saccades, and regressions) extracted from gaze, and typography as the medium through which interventions act. The review of related work positioned this thesis within the lineage of prior Reading the Reader projects [5, 6, 7] and within the broader literature on eye-tracking in reading research and on intervention design. The market survey then showed that no existing product combines real-time gaze sensing, automatic typographic adaptation, and researcher-facing experiment control in a single modular platform.

Taken together, the literature and the market landscape indicate that an adaptive reading system requires robust gaze-processing pipelines, carefully designed intervention mechanisms, measurable context-preservation outcomes, and a modular, experiment-ready architecture. The gap analysis distilled these into three concrete opportunities, concerning experimentability, the sensing layer, and the integration of external decision modules, which the final problem statement in Sec. 2.5 converts into the research questions that guide the remainder of the thesis. Chapter 3 describes the methodology adopted to address them.

3 Methodology

This thesis is conducted within the Design Science Research (DSR) paradigm, in which the construction and evaluation of an artifact is the central research contribution [8, 37]; Sec. 3.1 develops this orientation. The process by which that artifact was produced is organised using the Double Diamond model [38], a human-centred design framework introduced by the Design Council that structures a design project into two sequential diverge-then-converge cycles. The first diamond covers the problem space: an initial phase of wide exploration that opens up the inquiry, followed by a focusing phase that narrows the findings into a precise, actionable problem statement. The second diamond covers the solution space: a generative phase that develops and iterates on the solution, followed by a delivery phase that converges on a tested, evaluated artifact (in this thesis, delivering the artifact and evaluating it against the research questions). Fig. 3.1 illustrates this structure and maps each phase to the corresponding stage of this thesis.

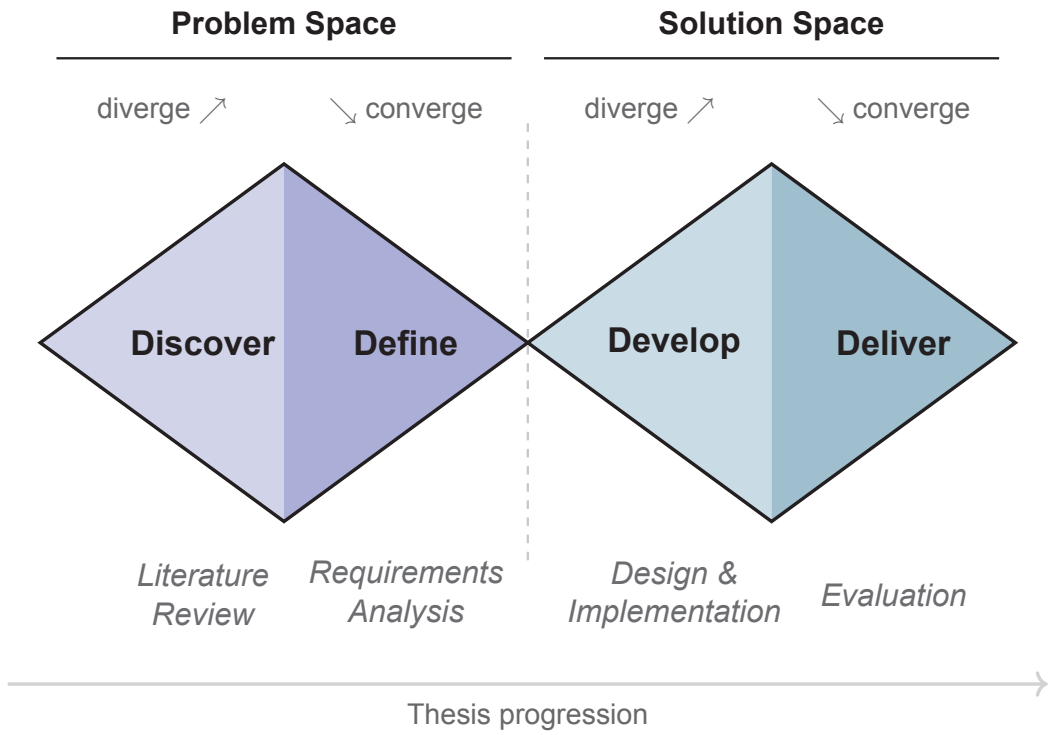


Figure 3.1: The Double Diamond process model [38] as applied in this thesis. Each diamond represents one diverge-then-converge cycle: the first covers the problem space (literature review and requirements analysis) and the second covers the solution space (design and implementation, then evaluation). Italic labels below each phase name indicate the corresponding thesis activity.

The remaining sections of this chapter detail each methodological dimension in turn. Sec. 3.1 describes the research approach that guides the four phases. Sec. 3.2 explains how requirements were derived. Sec. 3.3 describes the iterative engineering process and tooling. Sec. 3.4 sets out the evaluation strategy. Sec. 3.5 justifies the technology choices for each sub-project. Sec. 3.6 addresses ethical considerations arising from the use of eye-tracking data.

3.1 Research Approach

This thesis follows the Design Science Research paradigm: its primary contribution is a designed and implemented software artifact (a researcher-operated adaptive reading platform) rather than an empirical study of human reading behaviour [8, 37]. The research approach is therefore shaped by the nature of that contribution: the goal is not to measure a natural phenomenon but to construct, justify, and evaluate a designed system. As noted at the opening of this chapter, DSR and the Double Diamond operate at two different levels and are used together: DSR is the research paradigm, fixing what counts as a contribution and how it is evaluated [8], while the Double Diamond is the process organisation that sequences the work [38]. DSR does not by itself prescribe a single process model; its guidelines can be paired with a separate, complementary process, as Schorr and Hvam do when they apply DSR through a staged process with explicit evaluation steps [39, 37], and in this thesis the Double Diamond fills that role.

This paradigm determines what counts as a research output in this thesis. In DSR, design decisions and the artifact itself are first-class contributions that must be motivated, documented, and evaluated rather than merely implemented [8]. The architecture itself (the separation of sensing, decision-making, intervention execution, and researcher interface into independently replaceable modules) is the central claim this thesis makes, and the evaluation is designed to assess whether that claim holds in the implemented system. This orientation follows the design-science guidelines of Hevner et al.: the contribution takes the form of a viable artifact, specifically an instantiation; its utility and quality are to be demonstrated through well-executed evaluation methods rather than asserted; and its verifiable contribution lies in both the artifact and the design knowledge embodied in its module boundaries and integration contracts [8].

The research questions stated in section 2.5 follow directly from this paradigm. RQ1 through RQ4 all ask about properties of the designed system (modularity, sensing pipeline feasibility, context-preserving intervention, and researcher experimentability), as is characteristic of DSR, rather than about properties of human reading behaviour. This is a deliberate boundary: whether specific typographic interventions measurably improve reading outcomes for participants is a question for a future empirical study conducted with this platform, and it is explicitly outside the scope of this work. The platform is the instrument that makes such future studies possible; building and validating that instrument is the contribution of this thesis.

3.2 Requirements Elicitation

Requirements for this platform were elicited primarily through iterative discussions with the project supervisors, who as the primary stakeholders defined the research goals and experimental constraints that the platform must satisfy, rather than through up-front specification. The process is summarised in Fig. 3.2. Because a prototype of the adaptive reading system already existed at the start of this thesis work, those discussions were informed by a supporting retrospective analysis of that prototype alongside the original project proposal [3] and the findings of the literature review, which together helped surface what a research-capable platform must be able to do. This prototype was the codebase produced by the predecessor Reading the Reader student projects (section 2.2.1) [5, 6], which the literature review identified as capable but tightly coupled. Across successive supervision sessions an initial set of requirements was refined, with gaps and implicit assumptions surfaced and made explicit.

User stories were written from the perspective of the three primary actors (the researcher,

the participant, and the external module provider) and were used as the primary vehicle for communicating behavioural expectations across the team. Functional and non-functional requirements were then derived from those user stories, with the non-functional requirements specifically motivated by the architectural goals of the thesis: modularity, low latency, extensibility, and reproducibility. Each requirement was assigned a MoSCoW priority (Must have, Should have, Could have, or Won't have) [40], providing a principled basis for identifying the minimum viable capability set and for deferring lower-priority features when time pressure required trade-offs.

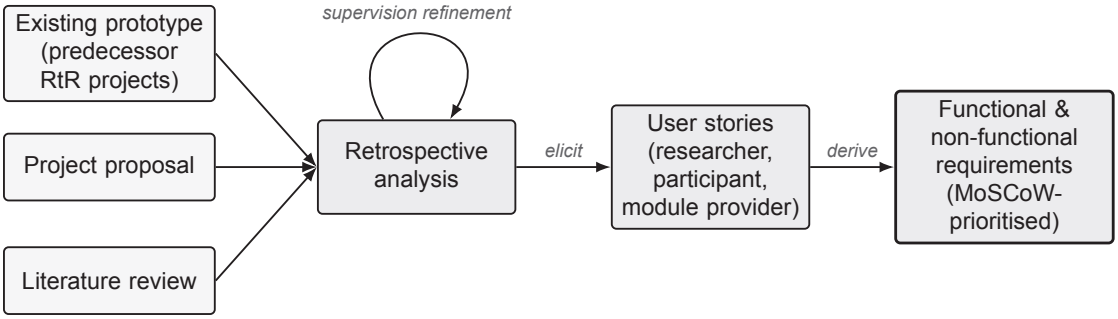


Figure 3.2: The requirements elicitation process. An existing prototype (the predecessor Reading the Reader projects), the project proposal, and the literature review were subjected to a retrospective analysis that informed iterative discussions with the supervisors, the primary elicitation activity, producing user stories for the three actors, from which MoSCoW-prioritised functional and non-functional requirements were derived.

We note that this elicitation approach is a deliberate consequence of the thesis type. Because the contribution is an artifact, the requirements serve a dual purpose: they establish what the system must do, and they provide the criteria against which the architectural design decisions in chapter 5 are later justified.

We also acknowledge a limitation of this approach. The requirements were validated chiefly through supervision and against the prior prototype, rather than through direct engagement with the researchers who will ultimately operate the platform. Deriving requirements from an existing artifact in this way risks inheriting that artifact's assumptions and overlooking needs that only a prospective operator would surface, a construct-validity limitation we revisit in chapter 7.

3.3 Development Process

The system was developed by a team of two as an iterative build, with each iteration targeting a coherent vertical slice of functionality (from sensing to decision to intervention to researcher interface) rather than building one layer at a time. This approach was chosen over a design-first waterfall process for two reasons. First, the integration between the Tobii hardware layer, the realtime transport, and the frontend intervention rendering could not be fully specified upfront: latency, timing, and synchronisation constraints only become visible under end-to-end execution. Second, a vertical-slice structure ensures that integration assumptions are tested and corrected at each increment rather than deferred to a final integration phase, reducing the risk of late-stage architectural surprises. This iteration takes place within the Develop phase of the second diamond: the Double Diamond describes the overall diverge-then-converge progression of the thesis, not a single linear pass through implementation, which instead advanced as repeated build-test-correct increments. Within that Develop phase, divergence took the form of exploring

design and integration options inside each vertical slice (rather than prototyping competing whole-system architectures in parallel), with the build-test-correct loop serving as the convergence mechanism that settled each slice on a single approach before it was considered complete.

The codebase is organised as a monorepo containing five sub-projects: the frontend application, the backend API and realtime server, the documentation site, a reference decision-maker, and a reference eye-movement analyser. This structure allowed shared conventions and contracts to be maintained across components without requiring a separate package registry, while keeping each sub-project independently buildable and deployable.

Version control was managed using Git with GitHub as the remote host. Changes were introduced through pull requests, providing a lightweight code-review step between the two authors before merging to the main branch. The CI pipeline configuration is described in section 3.5.

The full codebase is published on GitHub and released under the MIT License, making it freely available for inspection, reuse, and extension by future researchers and developers. This decision reflects the open-science orientation of the work: because the architecture and the integration contracts are themselves research contributions, making the source openly accessible is necessary for those contributions to be reproducible and verifiable by others.

3.4 Evaluation Strategy

The evaluation of this thesis addresses three distinct questions that together cover the four research questions of section 2.5: whether the system functions correctly as a platform for reading experiments, whether the architecture satisfies the modularity and extensibility goals that motivate the design, and whether its runtime behaviour meets the live-operation constraints that adaptive reading imposes. Tab. 3.1 maps each research question to the method by which it is evaluated, and the remainder of this section describes those methods.

For functional correctness, we evaluate the system against the functional requirements defined in chapter 4, verifying that each requirement is met through a combination of automated tests, manual end-to-end walkthroughs of the researcher and participant workflows, and inspection of exported session data. For architectural quality, we evaluate the degree to which the modular boundaries between sensing, decision-making, intervention execution, and user interface hold in the implemented system, and whether the external module provider integration contract is sufficient for a third-party provider to connect without modifying the core application. Concretely, the architectural-quality assessment summarised in the RQ1 row of Tab. 3.1 is carried out through four means:

1. Inspection of the dependency graph to confirm that no module imports across the boundaries defined in the design.
2. Demonstration that the external integration contract is sufficient for the reference decision-maker sub-project to connect to the core runtime without modifying it.
3. Review of the module installer pattern to confirm that new implementations can be substituted at the composition root without touching application logic.
4. Assessment of modification locality: how much of the core must change to add a new intervention or substitute a decision provider, moving the measure beyond the

Table 3.1: Mapping of each research question to its evaluation method. The reading-behaviour effect study that a full evaluation of intervention efficacy would require is outside the scope of this thesis.

Research question	Evaluation method
RQ1 — Modularity	Dependency-graph inspection (no imports across module boundaries), module-installer review, an assessment of how much of the core must change to add or replace a module, and demonstration that a reference provider connects without modifying the core.
RQ2 — Sensing pipeline	Measurement of end-to-end gaze-to-event and gaze-to-intervention latency under live operation, and confirmation that the analysis stage is inspectable, loggable, and replayable.
RQ3 — Context-preserving intervention	Demonstration that interventions are committed at controlled boundaries, together with inspection of the logged context-preservation events.
RQ4 — Researcher control	Demonstration of the researcher's session controls and verification of the auditable record through session replay and data export.

binary presence or absence of cross-boundary imports to a graded indication of how well the boundaries localise change.

The remaining methods in Tab. 3.1 concern the runtime behaviour of the platform rather than its structure: the gaze-to-event and gaze-to-intervention latency measurement for RQ2, and the boundary-commit and session-replay demonstrations for RQ3 and RQ4, are carried out as part of the evaluation reported in chapter 7.

The latency budget against which RQ2 is assessed is not arbitrary: it is set by the timescale of natural reading. Eye movements in reading unfold over fractions of a second, with individual fixations lasting on the order of a few hundred milliseconds [41], so an adaptation that is to remain in step with the reader must be applied well within that window. The precise budget is fixed as a non-functional requirement in chapter 4 and measured in chapter 7, where latency is reported as a distribution (including its tail) rather than a single average, because live adaptation is sensitive to worst-case delay and jitter and not to mean throughput alone.

We acknowledge a structural threat to validity: as a two-person team that both specified and built the system, we cannot assess our own requirements and architectural decisions with full objectivity, and verifying functional correctness against requirements we authored ourselves carries a risk of confirmation bias. We mitigate this in three ways. First, we lean on automated tests, which encode pass/fail criteria independently of subjective judgement, as the more objective component of the functional assessment. Second, we publish the codebase and integration contracts openly under the MIT License, making them available for independent inspection. Third, we use the reference decision-maker and reference eye-movement analyser to show that the external integration contract is sufficient to build a provider against without modifying the core. We note, however, that because these reference modules were built by the same team, they evidence the sufficiency and separability of the contract rather than constituting its independent validation; genuinely independent validation would require a third-party provider, which we leave to

future work.

We do not conduct a user study of reading behaviour within the scope of this thesis. The platform is designed to enable such studies, but evaluating the effectiveness of specific typographic interventions on reading comprehension or fatigue is a separate research question that requires a dedicated experiment with recruited participants and an approved study protocol. That question is explicitly outside the scope of this work, as discussed in chapter 1.

3.5 Tools and Technologies

The frontend application is built with Next.js 16 and React 19 using TypeScript 5 in strict mode. Next.js was selected for its App Router, which provides the route grouping needed to separate the researcher dashboard from the full-screen participant reading view without a second application. TypeScript strict mode was chosen because it catches contract mismatches between the frontend and backend at compile time, which is especially important given that the shared contracts between the two are maintained manually rather than through code generation. State management uses Redux Toolkit with RTK Query for server-side data fetching: RTK Query centralises caching and invalidation of experiment session state, ensuring consistency across components without ad hoc fetch calls. Tailwind CSS v4 provides utility-first styling that keeps style declarations co-located with component logic. The frontend is built and served using Bun 1.3, chosen as the package manager and build tool for its fast dependency installation and cold-start build times.

The backend is built on .NET 10 with C# using ASP.NET Core as the HTTP and WebSocket host. .NET 10 was selected for its mature asynchronous runtime, which is required to handle concurrent gaze-data streaming over WebSocket alongside synchronous REST requests without blocking. REST endpoints are implemented with FastEndpoints 8, which reduces controller boilerplate while retaining full access to the ASP.NET Core middleware pipeline. Session data is exported using CsvHelper 33 for type-mapped CSV serialisation alongside the built-in `System.Text.Json` for JSON, avoiding manual string formatting for structured experiment output. The Tobii Research SDK (`Tobii.Research.x64` 1.11) provides the hardware integration layer for eye-tracker communication and is a Windows-only dependency [24]; the backend CI pipeline therefore runs on a Windows runner to match the production environment.

Continuous integration is managed through two independent GitHub Actions pipelines: the frontend pipeline runs on Ubuntu using Bun and validates every change with a full production build; the backend pipeline runs on Windows, restores dependencies, builds the solution in Release configuration, and executes the xUnit test suite.

Reading content is authored in Markdown. PDF support was considered but explicitly scoped out in favour of a simpler, tokenisation-friendly format that allows reliable gaze-to-word mapping without the coordinate complexity of PDF rendering.

3.6 Ethical Considerations

Eye-tracking data is sensitive by nature: it captures where a person directs their attention, moment by moment, and can in principle be used to infer cognitive states, reading difficulties, or other personal characteristics beyond the scope of any individual experiment. Collecting such data from study participants therefore requires careful handling.

Within this platform, all session data (including raw gaze samples, demographic attributes, and comprehension results) remains under the direct control of the researcher who operates the system. No data is transmitted to external servers; all storage is local to the

machine on which the backend is running. Participant attributes collected at enrolment (name, age, sex, existing eye conditions, and reading proficiency) are stored only within the session record and are not shared outside the system. We note that one of these attributes, existing eye conditions, is information about a person's health, which the GDPR treats as a special category of personal data subject to stricter protection than ordinary personal data [42]; its collection should be limited to what a given study genuinely requires.

Because these attributes are directly identifiable, researchers deploying the platform in a real study should apply pseudonymisation where required by their study protocol, replacing identifying fields with participant codes before any data leaves the controlled research environment. Data should be retained only for the duration required by the study protocol and applicable regulations, and deleted thereafter.

More fundamentally, collecting a directly identifying name sits in tension with the principles of data minimisation and of data protection by design and by default, both of which the GDPR establishes [42]. We therefore argue that a platform of this kind should, in a future iteration, issue participant codes at enrolment rather than store identifying fields and rely on the researcher to pseudonymise them afterwards: designing the data model so that identifying information need never be collected is a stronger safeguard than removing it after the fact.

The development of this platform did not itself require an institutional ethics review, as no human participants were recruited and no personal data was collected during the engineering work described in this thesis. The ethics obligations described above therefore apply to researchers who deploy the platform in a future study involving real participants. Before conducting such a study, the researcher is responsible for obtaining informed consent from each participant, ensuring that participants understand what data is collected and how it will be used, and complying with applicable data protection regulations, including the General Data Protection Regulation (GDPR) [42] as it applies at DTU. The platform does not implement consent workflows directly; this responsibility rests with the researcher conducting the study.

3.7 Chapter Summary

This chapter established the methodological foundations of the thesis. The work was framed within the Design Science Research paradigm, which treats the constructed artifact and its architectural decisions as the primary research outputs. The Double Diamond model provided the process organisation, mapping the four phases of the research (literature review, requirements analysis, design and implementation, and evaluation) onto two sequential diverge-then-converge cycles covering the problem space and the solution space respectively. The engineering process was described as iterative and vertical-slice driven, with continuous integration, pull-request-based review, and open-source publication supporting reproducibility and extensibility. Requirements were shown to be derived through retrospective analysis of an existing prototype, refined through supervision sessions, and expressed as user stories and MoSCoW-prioritised requirements. The evaluation strategy was defined around three questions: functional correctness verified against the requirements, architectural quality assessed through dependency inspection, modification-locality assessment, and external contract validation, and runtime behaviour assessed through latency measurement and live-operation demonstrations; structural threats to validity arising from self-assessment by a two-person team were acknowledged and partially mitigated through automated testing and open publication of the codebase. Ethical responsibilities arising from the collection of gaze and participant data were identified and assigned to the researcher operating the platform. The requirements derived

through the process described here are presented in full in the following chapter.

4 Requirements and Analysis

This chapter establishes the requirements for the Reading the Reader platform through a structured derivation chain that connects project stakeholders to measurable system constraints. Requirements were elicited primarily through discussions with the project supervisors, who defined the research goals and experimental constraints that the platform must satisfy, and were refined iteratively as the scope of the implementation became clearer. Rather than listing requirements in isolation, this chapter traces the full reasoning path: stakeholders and system actors are identified first, their needs are captured as user stories, the stories are decomposed into use cases, and the use cases are finally translated into functional and non-functional requirements. This derivation chain makes explicit which design decisions are grounded in stakeholder need and which arise from architectural or technical constraints. Sec. 4.1 introduces the project stakeholders and the five system actors; Sec. 4.2 presents the elicited user stories; Sec. 4.3 develops the use case model; Sec. 4.4 presents the conceptual domain model; Sec. 4.5 lists the functional requirements; Sec. 4.6 covers the non-functional requirements; Sec. 4.7 identifies the fixed external constraints; and Sec. 4.8 summarises the chapter.

4.1 Stakeholders and System Actors

The primary stakeholder of this platform is the Reading the Reader research initiative at DTU Compute [3], which commissioned the system to support controlled studies of adaptive typographic interventions on reading behaviour. The initiative's supervisors served as the requirements authority throughout the project: they participated in the user story elicitation sessions described in Sec. 4.2, reviewed scope and priority, and provided the research constraints that shaped the architecture. A secondary stakeholder group consists of future research teams and external contributors who may extend the platform with new intervention strategies or decision algorithms; their interests motivate the modularity and documented integration protocol that run through the design.

Five actors interact directly with the system at runtime.

4.1.1 Researcher

The researcher is the primary human operator and the stakeholder around whose workflow the platform is principally organised. In the context of a controlled reading study, the researcher is responsible for the complete experimental lifecycle: preparing reading materials and experiment templates, enrolling participants and recording their demographic attributes, ensuring the sensing hardware is correctly connected and calibrated, and conducting the session from start to finish.

During an active session, the researcher operates from a physically separate screen, observing the participant's reading behaviour through a mirrored live view without being present at the participant's display. From this interface, the researcher must be able to monitor session health, manually apply typographic interventions, approve or reject proposals submitted by an automated decision strategy, and pause and resume automated decision-making. Once a session concludes, the researcher must be able to export session data for external analysis and replay recorded sessions to review gaze behaviour and intervention history at a later point.

Because the researcher coordinates every aspect of an experiment, minimising operator cognitive load is a primary concern. A guided, step-by-step setup workflow with explicit

readiness gating and a unified live control panel are therefore core requirements, not peripheral usability enhancements.

4.1.2 Participant

The participant is the person recruited by the researcher to take part in a reading experiment. Their role within the system is deliberately narrow: they read Markdown-formatted text presented in the reading interface while their gaze behaviour is recorded. The participant does not interact with system configuration, session management, or any researcher-facing controls.

During a session, the participant may be exposed to typographic adaptations (changes to font size, line width, line height, letter spacing, font family, colour palette, or theme mode) applied by the researcher or by an automated strategy. A critical requirement arising from this role is that such adaptations must not disrupt reading flow or cause the participant to lose their place in the text, as perceptible disruption would confound the experimental results. Following each reading item, the participant may also be asked to answer comprehension quiz questions, the results of which are recorded as part of the session data.

4.1.3 External Module Provider

The external module provider is a third-party process that connects to the platform to supply decision logic or eye-movement analysis without requiring modifications to the core system.

A central research motivation for this platform is the ability to evaluate different decision and analysis strategies under comparable experimental conditions. This actor motivates the need for a stable and documented integration protocol so that different strategies (rule-based, machine-learning-based, or otherwise) can be substituted without changes to the core system. The platform must support a range of execution modes, from fully autonomous application of proposals to researcher-gated approval, so that the degree of automation can itself be varied as an experimental condition. The system must also continue operating normally if a provider disconnects mid-session, ensuring that a provider failure does not abort an ongoing experiment.

4.1.4 Eye Tracker Device

The eye tracker device is the hardware sensing actor that supplies the raw gaze stream on which the platform's adaptive behaviour depends. In the context of this project the device is a Tobii screen-based eye tracker [24], though the system must support hardware sensing sources through a stable interface so that alternative devices can be substituted without changes to the application layer. The device streams gaze coordinates, per-eye validity signals, and timestamps at high frequency; it does not initiate any use cases of its own but must be detected, licensed, and calibrated before a session can begin. A key requirement arising from this actor is that the system must handle device disconnection and reconnection gracefully during a session so that a transient hardware failure does not abort an ongoing experiment.

4.1.5 Data Consumer

The data consumer represents the downstream actor (an analyst, a collaborator, or an automated analysis script) that processes the records produced at the end of a session. This actor does not interact with the running system; their interest is in the structure, completeness, and reproducibility of the exported artefacts. The requirement that follows is that every session export must be self-contained: it must bundle raw gaze samples, derived events, all intervention records with source attribution and timestamps, comprehension

results, and calibration metadata in a machine-readable format so that the full experimental sequence can be reconstructed without access to the live application. The data consumer also motivates the replay interface, which allows the same exported record to be loaded back into the platform for visual review.

4.2 User Stories

User stories were elicited through a series of structured discussions between the thesis authors and their supervisors, who represent the primary stakeholders identified in Sec. 4.1. Each story captures a unit of observable value from the perspective of a named actor, expressed in the conventional form *"as a [actor], I want [capability] so that [benefit]"*. The stories served as the primary elicitation artefact: they were reviewed with supervisors to confirm scope and priority, subsequently decomposed into the use cases presented in Sec. 4.3, and finally translated into the measurable functional requirements in Tab. 4.2.

Tab. 4.1 presents the elicited stories. Stories are grouped by actor and labelled for traceability.

Table 4.1: User stories elicited through stakeholder discussions, grouped by actor.

ID	Story
Researcher — Preparation	
US-R1	As a researcher, I want a guided step-by-step setup workflow so that I can prepare an experiment without missing critical steps.
US-R2	As a researcher, I want to create and manage reading materials with optional comprehension questions so that I can build controlled stimuli sets.
US-R3	As a researcher, I want to define reusable experiment templates so that I can run multiple sessions with consistent structure.
US-R4	As a researcher, I want to calibrate the eye tracker before each participant so that gaze data is spatially accurate for that individual.
US-R5	As a researcher, I want to select the sensing mode so that I can run demonstrations or pilot tests without physical eye-tracking hardware.
Researcher — Live Session	
US-R6	As a researcher, I want to observe the participant's reading view on a separate screen so that I can monitor behaviour without being visible to the participant.
US-R7	As a researcher, I want to see live gaze highlighting and a rolling attention heatmap so that I can assess reading focus in real time.
US-R8	As a researcher, I want to manually apply a typographic intervention at any point in a session so that I can act on observations the automated strategy does not capture.
US-R9	As a researcher, I want to approve or reject automated intervention proposals so that I retain control over what the participant experiences.
US-R10	As a researcher, I want to include a non-adaptive control condition so that I can compare adaptive and baseline reading experiences within the same platform.
US-R11	As a researcher, I want to compare two different decision strategies across sessions so that I can evaluate which produces better reading outcomes.
Researcher — Post-Session	
US-R12	As a researcher, I want to export session data in a structured format so that I can analyse gaze behaviour and intervention effects with external tools.
US-R13	As a researcher, I want to replay a recorded session so that I can review gaze paths and intervention history after the experiment concludes.
Participant	

Continued on next page

Table 4.1 — continued

ID	Story
US-P1	As a participant, I want typographic changes to be applied without disrupting my reading so that my natural reading behaviour is preserved for analysis.
US-P2	As a participant, I want my reading position to be preserved when the presentation changes so that I do not lose my place in the text.
US-P3	As a participant, I want to answer comprehension questions after each reading item so that my understanding of the material can be assessed.
External Module Provider	
US-E1	As an external module provider, I want to connect to the platform over a documented protocol so that I can supply decision logic without modifying the core system.
US-E2	As an external module provider, I want to receive structured context envelopes so that I can base decisions on accurate, up-to-date gaze and session state.
US-E3	As an external module provider, I want my disconnection to be handled gracefully so that an ongoing session is not interrupted by a provider failure.

4.3 Use Cases and Scenarios

Fig. 4.1 provides a complete overview of the system's use cases and their actor associations. The diagram uses standard UML use case notation: stick figures are actors, ovals are individual use cases, solid lines are associations, and dashed arrows labelled `<<include>>` indicate that one use case necessarily invokes another as part of its execution. The twenty-one use cases are organised into five horizontal bands that reflect the natural phases of system use: *Setup & Calibration*, *Content Management*, *Session Configuration*, *Live Session*, and *Post-Session*.

The system involves five actors. The **Researcher** is the primary operator and is associated with fourteen of the twenty-one use cases, spanning all five bands. The **Participant** is involved in three use cases: they fixate on calibration points during setup, read the presented text during the live phase, and answer comprehension questions at the end of each reading item. The **Eye Tracker Device** is a hardware actor that contributes a single high-frequency use case: streaming raw gaze samples into the pipeline. The **External Module Provider** is an out-of-process actor that connects to the platform over a dedicated integration interface to supply decision logic or eye-movement analysis. Finally, the **Data Consumer** represents any downstream stakeholder (analyst, collaborator, or analysis script) that accesses exported session records.

The following four scenarios trace representative paths through the diagram, covering all five bands and motivating the functional requirements in Sec. 4.5.

4.3.1 UC-1: Preparing an Experiment

This scenario traces the *Content Management* and *Setup & Calibration* bands of Fig. 4.1, covering all work a researcher does before a participant arrives.

The preparation begins with the reading library. Using *Manage Reading Materials*, the researcher creates one or more materials, each carrying a Markdown content body, descriptive metadata, and an optional set of comprehension questions. *Define Experiment Template* then assembles an ordered sequence of those materials into a reusable session plan; individual items may carry their own typographic defaults. Global session parameters (intervention policy, execution mode, and decision strategy selection) are fixed through *Configure Session Settings*.

Hardware setup follows. The researcher connects the Tobii eye tracker and uses *Detect &*

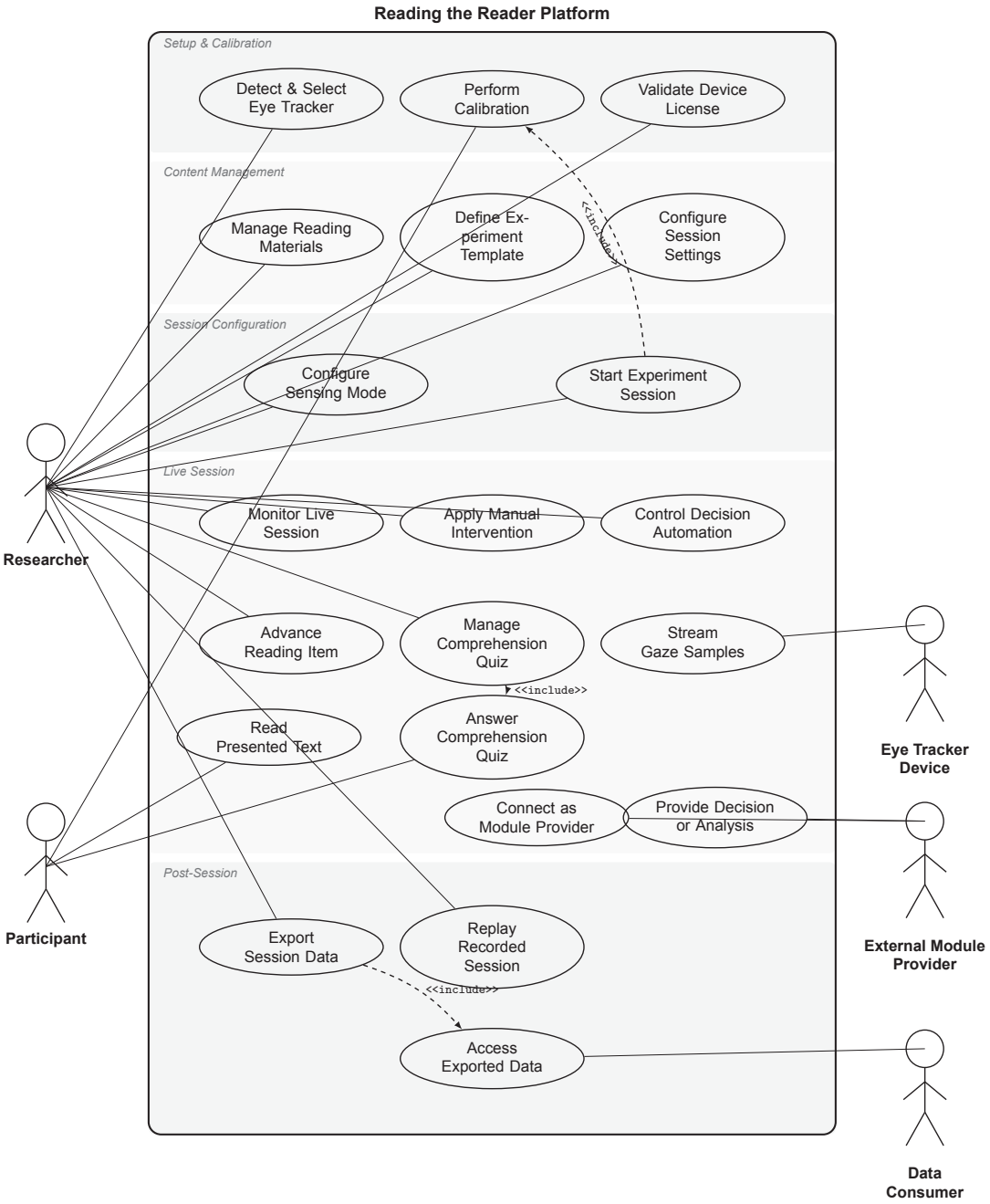


Figure 4.1: UML use case diagram for the Reading the Reader platform. Solid lines denote actor–use-case associations; dashed arrows denote `<<include>>` dependencies. Use cases are grouped into five bands: Setup & Calibration, Content Management, Session Configuration, Live Session, and Post-Session.

Select Eye Tracker to identify and activate the device. *Validate Device License* provisions the device licence before any gaze data is permitted to flow. *Perform Calibration* runs the calibration point-collection and validation workflow; the participant fixates on each calibration point in turn while the system records the correspondence between gaze signal and screen position. The researcher initiates and oversees this step rather than performing it, reviewing the resulting quality metrics and repeating the procedure if the accuracy threshold is not met. All three hardware steps must pass before the session-start control becomes available. This scenario motivates FR2 (Eye-Tracker Detection and Licensing), FR3 (Calibration Integration), FR5 (Reading Content and Presentation), FR14 (Reading Material Library), and FR15 (Experiment Setup Templates).

4.3.2 UC-2: Launching and Conducting a Live Reading Session

This scenario spans the *Session Configuration* and *Live Session* bands and involves three actors: the Researcher, the Participant, and the Eye Tracker Device.

The researcher selects the active sensing source via *Configure Sensing Mode*: a physical Tobii device for a real experiment, or mouse-cursor simulation when running a demonstration without hardware. *Start Experiment Session* then simultaneously opens the participant reading interface on a second screen and activates the researcher control panel. This use case carries a `<<include>>` dependency on *Perform Calibration*, enforcing that the calibration gate from UC-1 has been passed.

Once the session is live, the Eye Tracker Device continuously executes *Stream Gaze Samples*, feeding high-frequency gaze coordinates to the system in real time. The Participant executes *Read Presented Text* on their screen while the researcher uses *Monitor Live Session* to observe the mirrored view, live gaze highlighting, and a rolling attention summary.

The configured decision strategy processes the incoming gaze and eye-movement analysis signals. In automated mode it applies typographic changes directly; in hybrid mode it emits a proposal that the researcher acts on through *Control Decision Automation*. The researcher may also bypass the strategy entirely and apply an immediate change via *Apply Manual Intervention*. In all cases the context-preservation mechanism anchors the participant's reading position so that layout changes do not disorient them.

Where comprehension assessment is required, *Manage Comprehension Quiz* is activated; this use case carries a `<<include>>` dependency on *Answer Comprehension Quiz* on the participant's side, and response timing is recorded before the session continues. This scenario motivates FR1 (System Execution and Setup), FR4 (Gaze Data Acquisition and Mapping), FR6 (Researcher Interface), FR7 (Intervention Runtime), FR8 (Decision-Strategy Abstraction), FR9 (Context Preservation), FR11 (Experimental Control), FR13 (Comprehension Quiz), FR16 (Multi-Item Session Sequencing), FR17 (Sensing Mode Configuration), and FR20 (Decision-Proposal Approval Workflow).

4.3.3 UC-3: Integrating an External Module Provider

This scenario involves the External Module Provider actor and the two *Live Session* band use cases on the right side of Fig. 4.1: *Connect as Module Provider* and *Provide Decision or Analysis*.

A researcher wishes to evaluate a custom fixation-detection model or an AI-driven reading-difficulty classifier without modifying the core application. They launch a compatible external process and connect it to the platform's dedicated module-provider integration interface. Upon connection the provider executes *Connect as Module Provider*, declaring

its capabilities to the system: whether it processes raw gaze samples and returns fixation events, or whether it receives decision context and returns intervention recommendations.

For the remainder of the live session the provider executes *Provide Decision or Analysis* on an ongoing basis. Commands it returns pass through the same parameter-validation and intervention-logging path as built-in strategies, and therefore appear identically in the session record and in the researcher's control panel. If the provider disconnects unexpectedly the session is not interrupted: the platform logs the event and continues with the mode that was active before the provider connected. This scenario demonstrates that the extensibility requirement is fulfilled at the protocol level: new decision and analysis algorithms are delivered as separate processes communicating over a documented interface, and the core codebase is left unmodified. This scenario motivates FR18 (Eye-Movement Analysis Layer) and FR19 (External Module Provider Framework).

4.3.4 UC-4: Reviewing and Exporting Session Data

This scenario covers the remaining *Post-Session* use cases and involves both the Researcher and the Data Consumer actor.

When a session ends the researcher uses *Export Session Data* to produce a structured session record. This use case carries a `<<include>>` dependency on *Access Exported Data*: the export operation both writes the artefact to the server and immediately makes it available for retrieval. The record bundles raw gaze samples, derived fixation and saccade events, all intervention events with source attribution and timestamps, comprehension quiz results, calibration metadata, and the complete typographic configuration, in a structured machine-readable format.

The researcher may then load any saved export into the platform via *Replay Recorded Session*, which reconstructs the reading interface with animated gaze-path playback, intervention markers on the timeline, and variable-speed playback controls. Previously saved exports are listed in the application and are reloadable without manual file management, satisfying the need to review multiple sessions across a study.

Independently of the researcher, a Data Consumer executes *Access Exported Data* to retrieve the record for offline analysis. Because the export alone is sufficient to reconstruct the full sequence of sensing, analysis, and intervention events, this use case is the primary mechanism through which the reproducibility requirement is satisfied. This scenario motivates FR10 (Real-Time Performance Monitoring), FR12 (Logging and Export), FR21 (Session Replay Playback), and FR22 (Session State Recovery).

4.3.5 End-to-End Session Workflow

While the use case diagram in Fig. 4.1 captures actor associations, it does not express the order in which activities occur, the conditions under which the workflow branches, or the activities that proceed concurrently during a live session. Fig. 4.2 traces the control flow of a complete experimental session, synthesising the preparation path of UC-1 and the live path of UC-2 into a single workflow, partitioned into swimlanes that show which actor performs each activity. The diagram remains at the analysis level: every node denotes an activity, every branch a condition, and every lane an actor, with no reference to system components or internal mechanisms.

Two readiness gates structure the flow. Calibration repeats until its quality threshold is met, and a reading item cannot begin until calibration has passed and the device is licensed. During each reading item, three activities proceed concurrently: the participant reads the presented text, the system records gaze and derives fixation events, and the researcher monitors the live session. Within the sensing activity an intervention is applied

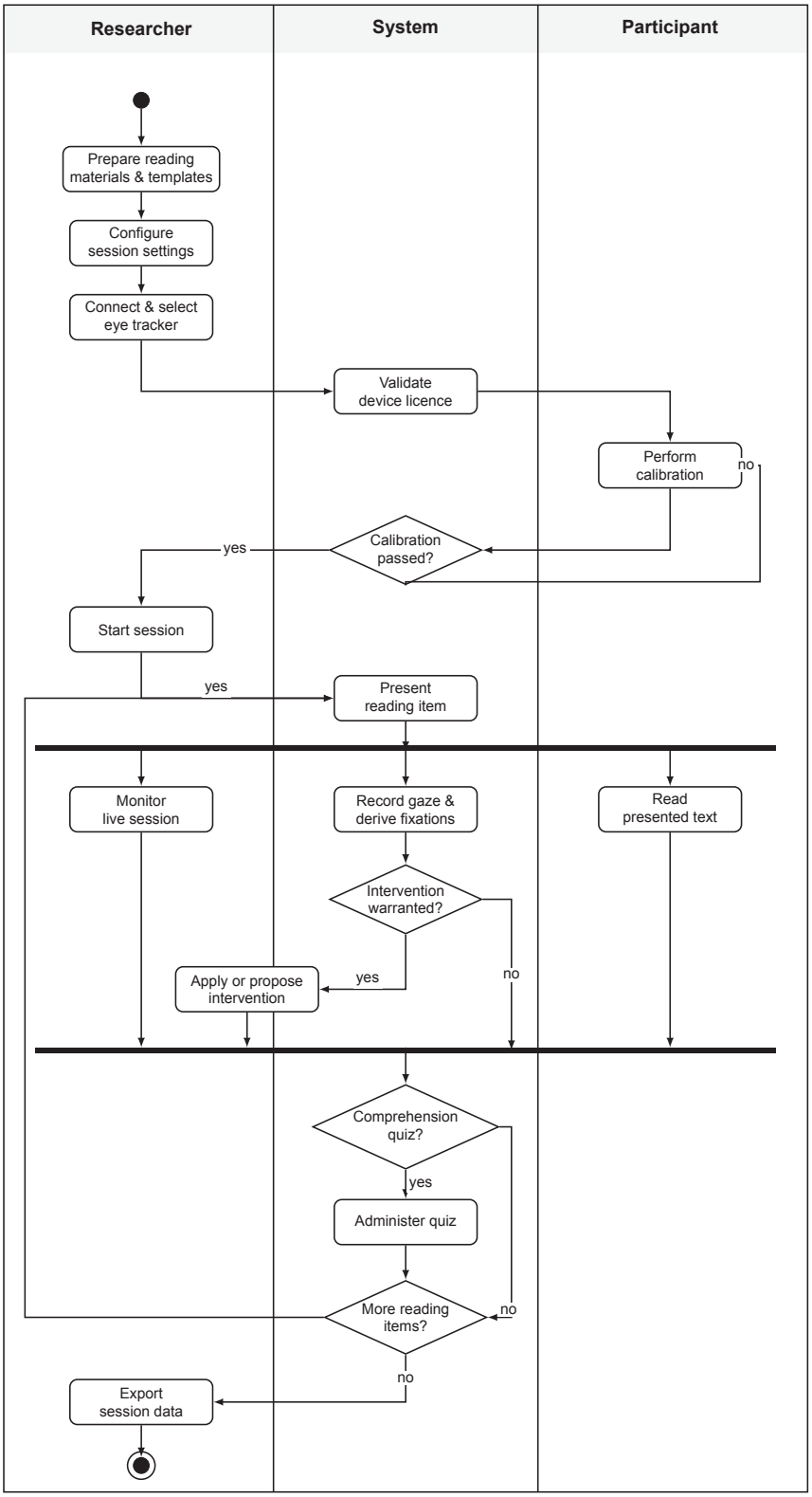


Figure 4.2: Swimlane activity diagram for a complete experimental session, partitioned by actor into Researcher, System, and Participant lanes. Rounded boxes are activities and diamonds are decisions; the solid horizontal bars denote the fork and join that bound the concurrent activities of the live reading phase. Each activity sits in the lane of the actor that performs it: the researcher prepares and operates the session, the participant performs the calibration and reads, and the system validates, senses, and adapts. The intervention activity straddles the researcher and system lanes because an intervention may be applied automatically by the system or proposed by the system and approved by the researcher. The diagram stays at the analysis level, describing what happens, under what conditions, and by whom, rather than how the system is structured internally.

or proposed only when the current reading state warrants it, reflecting the configurable execution modes captured in FR11. The outer loop returns to the next reading item until the experiment template is exhausted, after which the session data is exported for later analysis.

4.4 Domain Model

Fig. 4.3 presents the conceptual domain model for the Reading the Reader platform. The model identifies the principal entities in the problem domain, their key attributes, and the associations between them, expressed at the analysis level without reference to any implementation structure. It establishes the shared vocabulary that grounds the functional requirements stated in Sec. 4.5.

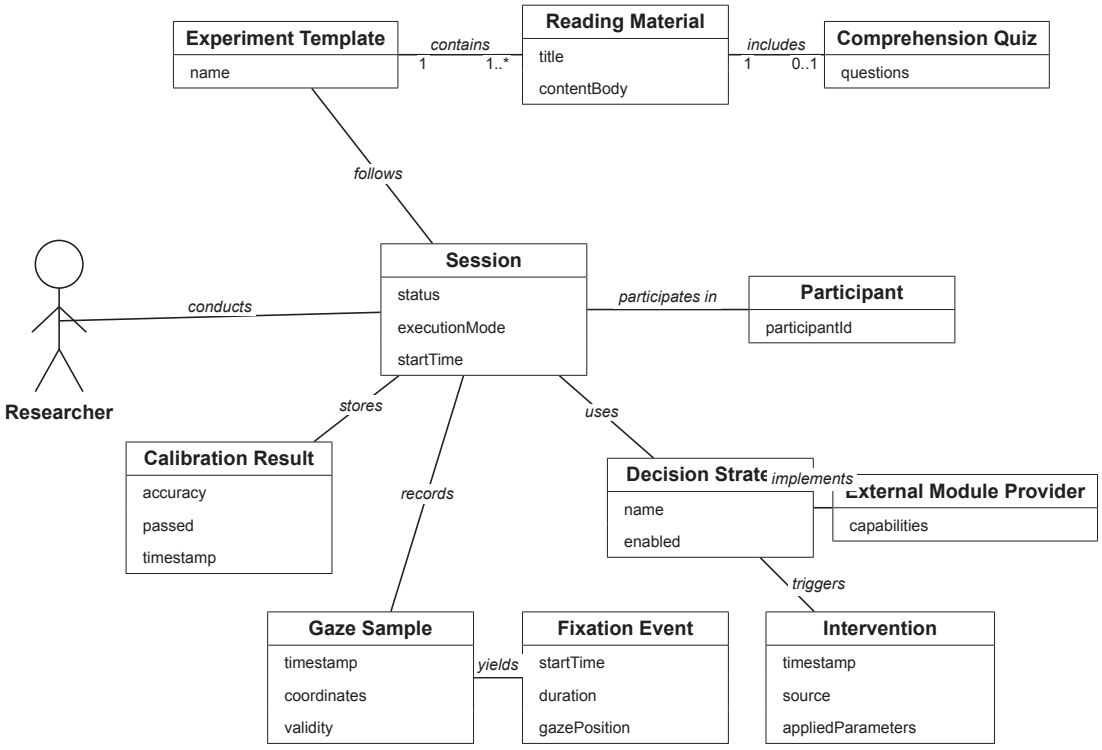


Figure 4.3: Conceptual domain model for the Reading the Reader platform. Each box shows the entity name and its analysis-level attributes. Lines represent associations labelled with the nature of the relationship; multiplicity notation follows UML convention. The Researcher is shown as a UML actor because it represents a human role with no data attributes at this level of analysis.

The model centres on **Session**, the primary unit of experimental activity, which carries the execution mode and status that determine how the session is conducted. A session is associated with one **Researcher** and one **Participant** (identified by a *participantId*) and follows a predefined **Experiment Template** that specifies an ordered set of **Reading Materials**. Each reading material carries a title and a content body, and may include an optional **Comprehension Quiz** with its associated questions. Before a session begins, calibration is performed and the outcome is stored as a **Calibration Result**, recording accuracy, pass or fail status, and a timestamp. During the session, the eye tracker produces **Gaze Samples** carrying timestamped coordinates and a validity flag; the analysis layer yields **Fixation Events** from these samples. A **Decision Strategy** (characterised by a name and an enabled flag) observes the incoming events and triggers **Interventions**,

each recorded with a timestamp, the source that triggered it, and the parameters applied. An **External Module Provider** may fulfil the strategy role by connecting to the platform and declaring its capabilities through the integration interface.

4.5 Functional Requirements

Tab. 4.2 lists the functional requirements for the system, grouped by concern and prioritised using the MoSCoW method [40], where **M** = Must Have, **S** = Should Have, and **C** = Could Have. Requirements labelled **M** are non-negotiable for the platform to fulfil its thesis contribution; those labelled **S** enhance experimental quality and researcher usability; those labelled **C** are optional enhancements that were implemented as an extension of the core design. Won't Have (W) items are not listed in this table; features explicitly excluded from scope are captured as constraints in Sec. 4.7. The requirements were derived from the use cases in Sec. 4.3 and subsequently reviewed with the project supervisors in their capacity as primary stakeholders.

Table 4.2: Functional requirements with MoSCoW priority.

ID	Requirement	Pri.
FR1 — System Execution and Setup		
FR1.1	The system shall be launchable through a single startup command.	M
FR1.2	The system shall guide the researcher through a structured, multi-step setup workflow with explicit progress indicators.	M
FR1.3	The system shall prevent session start until device selection, license validation, and calibration are all completed successfully.	M
FR2 — Eye-Tracker Detection and Licensing		
FR2.1	The system shall detect connected Tobii eye trackers dynamically on startup and on reconnection.	M
FR2.2	The system shall allow the researcher to select exactly one active device from the detected set.	M
FR2.3	The system shall require a valid device license before enabling gaze data streaming.	M
FR2.4	The system shall handle device disconnect events gracefully without crashing or losing session data.	M
FR3 — Calibration Integration		
FR3.1	The system shall integrate the eye tracker's calibration workflow, including point collection and validation phases.	M
FR3.2	The system shall display calibration quality metrics and indicate pass or fail to the researcher before session start.	M
FR3.3	The system shall store calibration metadata in the session record for reproducibility.	M
FR3.4	The system shall allow recalibration between participants without restarting the application.	M
FR3.5	The system shall support selectable calibration point presets with varying point densities.	S
FR4 — Gaze Data Acquisition and Mapping		
FR4.1	The system shall stream gaze samples in real time from the active sensing source.	M
FR4.2	The system shall convert normalised gaze coordinates into screen-space coordinates.	M

Continued on next page

Table 4.2 — continued

ID	Requirement	Pri.
FR4.3	The system shall map gaze coordinates to content tokens (word-level granularity) in the reading interface.	M
FR4.4	The system shall expose gaze-to-content mapping events via a stable internal interface to downstream analysis and decision layers.	M
FR5 — Reading Content and Presentation		
FR5.1	The system shall load and render Markdown-formatted documents; PDF support is explicitly out of scope.	M
FR5.2	The system shall support at least the following typographic dimensions: font family, font size, line width, line height, letter spacing, colour palette, and theme mode.	M
FR5.3	The system shall allow the researcher to lock the typographic configuration for a session so that participant-side changes are prevented.	M
FR5.4	The rendering engine shall maintain stable, consistent coordinate references so that gaze-to-content mapping remains valid after presentation changes.	M
FR6 — Researcher Interface (Second Screen)		
FR6.1	The system shall provide a live mirrored view of the participant's reading interface, updated in real time.	M
FR6.2	The researcher interface shall display the current gaze-validity rate as a health indicator.	M
FR6.3	The researcher shall be able to apply any registered intervention module manually from the control panel at any time during a session.	M
FR6.4	The researcher interface shall display a chronological log of all intervention events applied during the current session.	S
FR6.5	The researcher interface shall surface pending decision proposals and provide approve and reject controls for hybrid execution mode.	S
FR7 — Intervention Runtime		
FR7.1	The system shall support pluggable intervention modules that are discoverable and invokable without modifying the core runtime.	M
FR7.2	New intervention modules shall be addable without modifying any existing module or the core runtime.	M
FR7.3	Each intervention module shall declare its supported parameters and validate inputs before execution.	M
FR7.4	The runtime shall support manual, automated, and hybrid (proposal-gated) triggering of interventions.	M
FR7.5	The system shall provide at least the following built-in intervention modules: font family, font size, line width, line height, letter spacing, theme mode, colour palette, and participant-edit lock.	M
FR8 — Decision-Strategy Abstraction		
FR8.1	The system shall support interchangeable decision strategies selectable per session.	M
FR8.2	The system shall allow enabling and disabling a strategy without restarting the application.	M
FR8.3	The system shall log the origin, parameters, and rationale of each intervention event.	M
FR8.4	The system shall provide a built-in rule-based decision strategy configurable via session settings.	M
FR8.5	The system shall support an external decision strategy that delegates intervention decisions to a connected module provider.	M

Continued on next page

Table 4.2 — continued

ID	Requirement	Pri.
FR9 — Context Preservation		
FR9.1	During a layout-changing intervention the system shall identify the participant's current reading position and preserve it across the presentation change.	M
FR9.2	The reading view shall adjust automatically so that the participant's identified reading position is restored to the same visual location after the change is applied.	M
FR9.3	Context-preservation failures shall be logged in the session record.	M
FR10 — Real-Time Performance Monitoring		
FR10.1	The gaze-validity rate shall be computed continuously and made available to the researcher interface without perceptible delay.	S
FR11 — Experimental Control		
FR11.1	The system shall support at least four configurable execution modes: control (no intervention), manual-only, automated-only, and hybrid (automated with researcher-approval gate).	M
FR11.2	The active execution mode and any mid-session changes to it shall be recorded in the session log.	M
FR11.3	The researcher shall be able to pause and resume automated decision-making at any point during a live session.	S
FR12 — Logging and Export		
FR12.1	The system shall log all raw gaze samples with timestamps and validity flags.	M
FR12.2	The system shall log derived eye-movement events (fixations and saccades) produced by the analysis layer.	M
FR12.3	The system shall log all intervention events with timestamp, source (manual, rule-based, or external provider), and applied parameters.	M
FR12.4	The system shall export session data in both JSON and CSV formats.	M
FR12.5	Exported data shall include schema versioning, session metadata, calibration results, and presentation configuration.	M
FR12.6	Previously saved exports shall be listable and reloadable from within the application without manual file management.	S
FR13 — Comprehension Quiz		
FR13.1	Reading materials shall support an attached set of multiple-choice comprehension questions defined at setup time.	S
FR13.2	The system shall present the quiz to the participant at the end of a reading item, under researcher control.	S
FR13.3	Quiz answers, correctness, and response timing shall be recorded in the session export.	S
FR13.4	The researcher shall be able to navigate quiz questions and manage the quiz lifecycle from the control panel.	S
FR14 — Reading Material Library		
FR14.1	The system shall maintain a persistent library of named reading materials accessible through the researcher UI.	S
FR14.2	The researcher shall be able to create, read, update, and delete reading materials without editing files directly.	S
FR14.3	Each reading material shall carry a title, a description, a Markdown content body, and an optional comprehension quiz.	S

Continued on next page

Table 4.2 — continued

ID	Requirement	Pri.
FR15 — Experiment Setup Templates		
FR15.1	The system shall allow researchers to define named experiment templates specifying an ordered sequence of reading items.	S
FR15.2	Each item in a template shall reference one reading material and may carry item-specific typographic configuration.	S
FR15.3	Templates shall be persisted and reusable across multiple sessions.	S
FR16 — Multi-Item Session Sequencing		
FR16.1	A reading session may contain more than one reading item drawn from an experiment template.	S
FR16.2	Each reading-item transition shall be recorded as a timestamped event in the session log.	S
FR17 — Sensing Mode Configuration		
FR17.1	The system shall support at least two gaze-sensing modes: Tobii eye tracker and mouse-cursor simulation.	M
FR17.2	The researcher shall be able to switch the active sensing mode without restarting the application.	M
FR17.3	Mouse-cursor simulation shall produce gaze data compatible with all system capabilities so that the full platform can be operated and tested without physical eye-tracking hardware.	M
FR18 — Eye-Movement Analysis Layer		
FR18.1	The system shall derive fixation, saccade, and regression events from the raw gaze stream using a built-in algorithm.	M
FR18.2	The system shall support replacement of the built-in fixation detector with an external analysis provider connected via the module-provider framework.	M
FR18.3	Fixation, saccade, and regression data produced by the analysis layer shall be made available to decision strategies and included in the session export.	M
FR19 — External Module Provider Framework		
FR19.1	The system shall expose a dedicated integration interface through which external processes can connect and register as module providers.	M
FR19.2	A provider shall declare its capabilities on connection; the system shall route the appropriate context envelopes to it and apply its commands through standard validation and logging paths.	M
FR19.3	A provider disconnection shall not interrupt the running experiment; the affected strategy shall degrade gracefully and the event shall be logged.	M
FR19.4	The module-provider protocol shall be documented so that third parties can implement compatible providers independently.	M
FR20 — Decision-Proposal Approval Workflow		
FR20.1	A decision strategy shall be configurable to emit proposals for researcher approval rather than applying interventions directly.	S
FR20.2	Pending proposals shall be surfaced to the researcher in the control panel with approve and reject controls.	S
FR20.3	The researcher shall be able to switch the execution mode between fully automated and proposal-gated at any point during a session without reconfiguring the strategy.	S

Continued on next page

Table 4.2 — continued

ID	Requirement	Pri.
FR21 — Session Replay Playback		
FR21.1	The system shall reconstruct a recorded session in the reading interface from a saved export file.	S
FR21.2	Replay shall support play/pause and at least three configurable playback speed levels.	S
FR21.3	The researcher shall be able to scrub the replay timeline and navigate between recorded key events.	S
FR21.4	Applied intervention events shall be marked on the replay timeline.	S
FR22 — Session State Recovery		
FR22.1	The system shall periodically persist the current session state to durable storage during an active experiment.	S
FR22.2	On startup, the system shall detect an interrupted session and offer the researcher the option to restore it.	S
FR23 — Per-Context Reader Shell Settings		
FR23.1	The system shall maintain independent typographic and appearance settings for the participant reading view, the researcher mirror view, and the replay view.	S
FR23.2	Changes to settings in one context shall not affect any other context.	S
FR23.3	Per-context settings shall be persisted across sessions.	S
FR24 — Facial State Sensing (Optional)		
FR24.1	When a webcam is connected and enabled, the system shall capture facial state observations (not gaze) and make them available to decision strategies as supplementary context.	C
FR24.2	Facial state sensing shall be strictly optional; no other system function shall depend on it.	C

Two requirements in Tab. 4.2 do not trace to a single use-case scenario but emerged during design as refinements of the platform's structure. FR23 (Per-Context Reader Shell Settings) follows from the second-screen arrangement: because the participant reading view, the researcher mirror, and the replay view are distinct surfaces (Sec. 4.1.1), each requires independent typographic settings so that adjusting one (for example, enlarging the researcher's mirror) does not alter what the participant sees. FR24 (Facial State Sensing) is an optional capability afforded by an attached webcam, supplying supplementary facial-state context to decision strategies; it is prioritised Could-have precisely because no other system function depends on it.

4.6 Non-Functional Requirements

Tab. 4.3 presents the non-functional requirements that govern the overall quality attributes of the system. They were derived from the architectural goals stated in the problem statement and from the operational constraints imposed by a researcher-run eye-tracking experiment. Priorities follow the same MoSCoW scheme [40] as Tab. 4.2; not all non-functional requirements are equally critical to the core thesis contribution.

Table 4.3: Non-functional requirements with rationale and MoSCoW priority.

ID	Quality attribute	Requirement	Rationale	Pri.
NFR1	Modularity	The system shall maintain strictly separate bounded layers for sensing, eye-movement analysis, decision strategies, intervention execution, researcher interface, and session management. Each boundary shall be expressed as an interface in the application layer; concrete implementations shall be substitutable without changes to the core.	Motivated by the future research teams stakeholder (Sec. 4.1); the modularity claim must be demonstrable by substituting components at test time.	M
NFR2	Low Latency	The system shall keep end-to-end gaze-to-intervention latency below 100 ms, the threshold at which an interface response ceases to be perceived as instantaneous [43, 44], and well within the duration of a single reading fixation (typically 200–300 ms) [41]. High-frequency gaze data shall be handled through a dedicated real-time channel, separate from general request-response communication.	Micro-interventions must be applied while the trigger condition is still present; a change delivered after the fixation that triggered it has ended would no longer match what the reader is looking at, invalidating the intervention rationale. The 100 ms ceiling is the conservative bound from interface-response perception, while the fixation timescale is the stricter reading-specific target.	M
NFR3	Usability	The system shall reduce experimenter cognitive load through a guided multi-step setup workflow, explicit readiness gating, and a unified control panel for all live session actions.	See Researcher actor (Sec. 4.1.1); minimising operator cognitive load is identified there as a core requirement, not a peripheral usability enhancement.	S
NFR4	Reliability	The system shall fail gracefully on eye-tracker disconnection, invalid license, gaze-stream interruption, and external-provider disconnection: logging the failure, surfacing a status indicator to the researcher, and continuing in a degraded mode where possible.	An experiment session cannot be re-run if a crash causes data loss or disturbs the participant mid-reading.	M

Continued on next page

Table 4.3 — continued

ID	Quality attribute	Requirement	Rationale	Pri.
NFR5	Reproducibility	All session parameters, calibration metrics, intervention settings, decision-strategy configuration, and system version information shall be captured in the session export so that a session can be understood and reproduced from that record alone.	See Data Consumer actor (Sec. 4.1.5); their need for self-contained, complete session records is the direct source of this constraint.	M
NFR6	Extensibility	Adding new intervention modules, decision strategies, or eye-movement analysis algorithms shall require only additive code changes and shall not require modification of existing modules or the core layer. External processes shall be connectable as module providers without any changes to the codebase.	See External Module Provider actor (Sec. 4.1.3); the stable integration protocol requirement is motivated there.	M
NFR7	Recoverability	The system shall periodically persist session state to disk during an active experiment. An interrupted session shall be detectable and restorable on the next startup without manual intervention by the researcher.	Hardware and software faults during a live session with a human participant are disruptive; even partial data recovery preserves the experimental record.	M
NFR8	Hardware Independence	The system shall be fully operable without a physical eye tracker by using mouse-cursor simulation or another software sensing mode, with no code changes required.	Development, CI verification, and demonstrations must be possible on machines without Tobii hardware attached; this property also enables testing of all architectural layers in isolation. It is the quality-attribute counterpart of FR17.3, which mandates the same capability at the functional level.	M

4.7 Constraints

This section identifies the fixed external limitations that bound the design and deployment of the platform. Unlike non-functional requirements, which are quality targets the system is designed to meet, constraints are given conditions that the system must accept without modification.

C1 — Hardware and Operating System

Real experimental sessions require the Tobii eye tracker to be connected to a Windows host. The Tobii Research SDK [24], which provides the gaze data acquisition API used by the sensing layer, is distributed as a Windows-only binary with no Linux or macOS equivalent. Mouse-cursor simulation is available on any platform and covers development,

automated testing, and demonstration scenarios, but it does not substitute for calibrated Tobii hardware in a controlled experiment context. A connected webcam supplies facial-state observations only (Sec. 4.5, FR24) and is not a gaze-sensing source.

C2 — Reading Content Format

The platform accepts reading material exclusively in Markdown format. PDF, HTML, LaTeX, and other rich document formats are out of scope. This constraint was agreed with the project supervisors and simplifies both the content authoring workflow and the gaze-to-content mapping layer, which requires a predictable, reflowable text structure.

C3 — AI and Decision Logic Scope

End-to-end AI decision logic is outside the scope of this implementation. The system exposes a documented integration protocol through which external processes can connect as module providers and supply decision recommendations or eye-movement analysis results. The architectural responsibility of this thesis is to make such integration possible and well-defined; the algorithms themselves are the domain of future contributors or external research teams.

C4 — Ethics and Data Privacy

Gaze coordinates, session metadata, and comprehension responses constitute personal data under the General Data Protection Regulation (GDPR). The system does not implement participant consent collection; this is the researcher's procedural responsibility and is handled outside the application prior to each session, typically through a written consent form. The researcher is also responsible for data retention, anonymisation, and disposal in accordance with applicable institutional and regulatory requirements.

4.8 Summary

This chapter established the full requirements basis for the Reading the Reader platform through a structured derivation chain. The project stakeholders and five system actors were identified, and their needs were captured as nineteen user stories elicited through discussions with the project supervisors. The user stories were decomposed into a twenty-one use case model spanning five operational phases, and four narrative scenarios traced the principal paths through that model. From the use cases, functional requirements were derived and prioritised using MoSCoW, covering sensing, calibration, content management, live session control, intervention execution, module provider integration, and post-session export. Eight non-functional requirements were stated across the quality attributes of modularity, latency, usability, reliability, reproducibility, extensibility, recoverability, and hardware independence, and four constraint groups (C1–C4) were identified governing hardware platform, content format, AI scope, and data privacy. Taken together, the stakeholder-to-use-case-to-requirement derivation chain constitutes the analysis of the problem: it makes explicit which design decisions are grounded in stakeholder need and which arise from external constraints, providing a traceable rationale for every architectural choice developed in the following chapters. The requirements established here form the specification against which the system design presented in Chapter 5 is developed and justified.

5 System Design and Architecture

This chapter presents the architecture of the Reading the Reader platform, which is the central contribution of this thesis. At its core the platform separates adaptive reading into four independently replaceable concerns (sensing, eye-movement analysis, decision, and intervention), joined by a bounded real-time loop and backed by a single authoritative session record. The chapter develops this architecture from the top down, beginning with the forces that shaped it and descending through progressively more detailed views to the seams through which it may be extended. Consistent with the design-science orientation set out in section 3.1, the emphasis throughout is on justifying these structures against the requirements of chapter 4 rather than merely describing them, so that the chapter answers the research questions of section 2.5 as it proceeds. Section 5.1 isolates the architecturally significant requirements as a set of design drivers; section 5.2 fixes the overall shape of the system and the architectural style it follows; section 5.3 develops the four-layer modular decomposition that carries the central claim; section 5.4 sets the four concerns in motion as the real-time adaptive loop; section 5.5 describes the authoritative session record and the reproducibility it affords; section 5.6 shows how the architecture is opened for extension through its seams; and section 5.7 consolidates the design decisions and their rationale.

5.1 Architectural Drivers

The preceding chapter specified what the platform must do, but not every requirement shapes its structure to the same degree. A recognised observation in software architecture is that only a subset of requirements is *architecturally significant*, in the sense of having a measurable and lasting effect on the architecture, and that identifying this subset is a necessary step in design [45]. This section isolates that subset for the Reading the Reader platform and treats it as the set of *architectural drivers* that the remainder of this chapter is designed to satisfy. Consistent with the Design Science Research orientation established in section 3.1, we make these drivers explicit for two reasons: they motivate the design decisions recorded in section 5.7, and they become the criteria against which the artifact is evaluated in chapter 7 [8]. The drivers are drawn from the Must-have requirements of sections 4.5 and 4.6, prioritised using the MoSCoW scheme [40], together with the fixed constraints of section 4.7; they are summarised in table 5.1.

The dominant driver is modular separability. The central claim of this thesis, set out in section 3.1, is that sensing, eye-movement analysis, decision strategy, and intervention execution can be made independently replaceable; NFR1 and NFR6 (table 4.3) restate this as binding quality requirements, and the Future Research Teams and External Module Provider stakeholders (section 4.1) are the parties who depend on it. This driver requires that every boundary between concerns be expressed as an interface, that implementations depend inward on those interfaces rather than on one another, and that a new intervention, analysis algorithm, or decision provider be addable through additive code alone. It is the force from which the architectural style of section 5.2 is derived, and it corresponds directly to RQ1.

How that separability may be achieved is constrained by the live, real-time nature of a reading session. The adaptive loop operates under a responsiveness budget (NFR2): high-frequency gaze data must travel a dedicated channel, and the indirection introduced for modularity must not delay an intervention past the point at which it remains valid.

Because a session involves a human participant and cannot be repeated without cost, the platform must also be robust in live operation (NFR4 and NFR7): it must degrade gracefully when a device or an external provider disconnects, and it must checkpoint session state so that an interrupted session is recoverable. These two drivers stand in tension with the first, since the cleanest separation of concerns is rarely the fastest or the most fault-tolerant at runtime, and reconciling them is a recurring theme of the design decisions in section 5.7.

A final driver belongs to this live cluster, concerning not how an intervention is delivered but what it must preserve once applied. Because a typographic micro-intervention changes the layout the participant is actively reading, the platform cannot commit it arbitrarily: it must apply the change at a controlled boundary and restore the reader's position across it, so that adaptation does not cost the participant their place (FR9). This context-preservation requirement is the criterion that RQ3 puts to the test, and it shapes the intervention contract introduced in section 5.3 and the commit discipline developed in section 5.4.

A further group of drivers follows from the platform's role as a research instrument rather than an end-user product. Reproducibility (NFR5) requires a single, schema-versioned session record that captures enough of each session, including its parameters, calibration, events, and presentation configuration, to reconstruct it from the record alone, which is the need expressed by the Data Consumer stakeholder (section 4.1). Hardware independence (NFR8) requires a sensing-source abstraction that admits a simulated, mouse-cursor-driven source alongside the Tobii device, enabling development, automated testing, and demonstration without hardware, and as a side effect allowing each architectural layer to be exercised in isolation. Because the platform is researcher-operated, it must additionally lower operator cognitive load (NFR3) through a guided setup workflow, explicit readiness gating, and a unified control surface, which shapes the researcher-facing structure described in section 5.3.

Alongside these drivers, four fixed constraints (section 4.7) bound the design space without being objectives to optimise. The Tobii Research SDK is distributed as a Windows-only binary (C1), so platform-specific code must be quarantined behind the sensing adapter rather than allowed to leak into the core. Reading material is restricted to Markdown (C2), whose predictable, reflowable structure is what makes a stable gaze-to-content coordinate mapping feasible. End-to-end decision intelligence is out of scope (C3), and this constraint is the direct origin of the external-provider seam and the decision-strategy abstraction: the architectural responsibility of this thesis is to make such integration well-defined, not to supply the algorithms. Finally, gaze data and comprehension responses are personal data under the GDPR (C4), which bounds what the session record captures and exports while leaving participant consent as a procedural matter outside the application.

Table 5.1: Architectural drivers, their origin in the requirements of chapter 4, the structural response each demands, and where each is evaluated. Driver identifiers are introduced here for reference in later sections.

ID	Driver	Origin	Architectural implication	Evaluated in
D1	Modular separability	NFR1, NFR6; RQ1	Interface seam per concern; inward dependency; additive extension	section 5.3; RQ1 (table 3.1)
D2	Real-time responsiveness	NFR2; FR4; RQ2	Dedicated real-time channel; bounded end-to-end loop latency	section 5.4; RQ2 (table 3.1)
D3	Live-operation robustness	NFR4, NFR7	Graceful degradation on disconnect; session-state checkpoint and restore	section 5.4; functional verification
D4	Reproducibility	NFR5; FR12, FR21; RQ4	Authoritative, schema-versioned session record; session replay	section 5.5; RQ4 (table 3.1)
D5	Hardware independence	NFR8; FR17	Sensing-source port admitting the Tobii device and a simulated (mouse-cursor) source	section 5.3; operation on simulated input
D6	Operator-centred control	NFR3; FR1, FR6, FR11; RQ4	Guided setup, readiness gating, unified control plane, second-screen mirror	sections 5.3 and 5.6; workflow walkthrough
D7	Context-preserving adaptation	FR9; RQ3	Two-phase (validate then commit) intervention contract applied at a controlled boundary that restores the reader's position	sections 5.4 and 5.6; RQ3 (table 3.1)

5.2 Architectural Approach and System Overview

The drivers of section 5.1 specify the forces the architecture must reconcile, but not the form it should take. This section fixes that form in two steps, moving from the system in outline to the style that governs its internal structure, before any single concern is opened in section 5.3. We first present the platform at two levels of abstraction, the system in its environment and the runtime containers it comprises, and then name and justify the architectural style that the dominant driver D1 demands.

At the highest level of abstraction the platform is a single, researcher-operated system that mediates a participant's reading session, shown in its environment in fig. 5.1. Four external actors surround it (section 4.1): the researcher operates the session, the participant reads, the Tobii eye-tracking device supplies the gaze signal, and an external decision provider supplies the adaptive intelligence that this thesis treats as out of scope (C3, section 4.7). We adopt the C4 model [46] for this view and the next, because its context and container levels match the two altitudes at which we wish to reveal the system. This outermost view fixes the system boundary, and with it the division of responsibility that the rest of the chapter elaborates: the platform owns the capture of gaze, its analysis, the execution of interventions, and the session record, whereas it deliberately owns neither the internals of the eye-tracking hardware nor the decision intelligence behind the

provider seam.

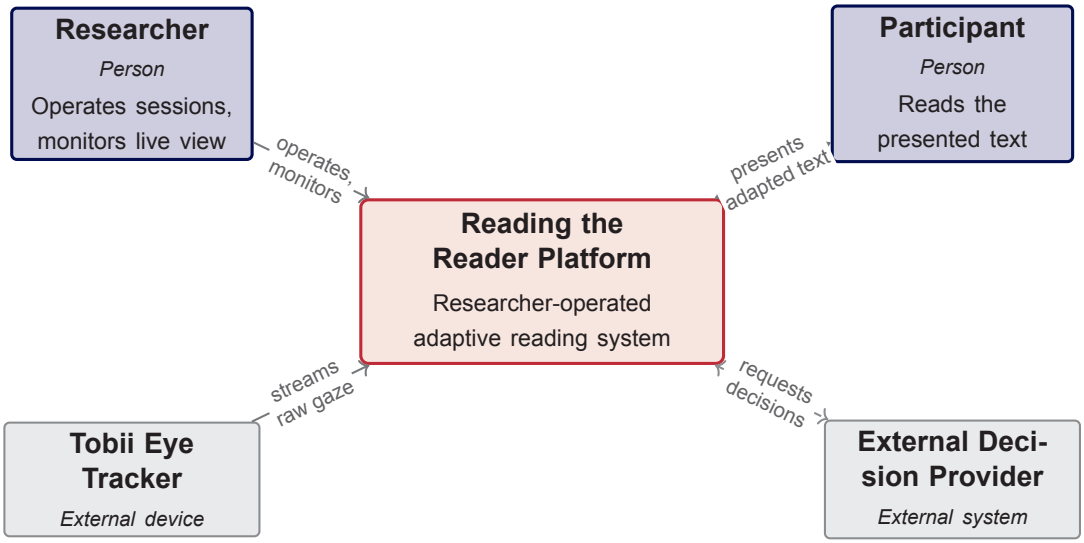


Figure 5.1: System context for the Reading the Reader platform, showing the researcher-operated system and the four external actors it mediates between.

Resolving the platform one level further reveals the runtime containers it comprises, shown in fig. 5.2. A researcher-facing control surface and a participant-facing reading surface are presented as two distinct surfaces of a single web front-end rather than a single shared screen, so that the participant sees only the text while the researcher retains a mirror of that view together with the controls around it (D6). A backend application host carries the core logic and mediates between the two surfaces, the hardware, and the store. Two channels connect the front-ends to the host: a request and response channel carries discrete operator commands and setup data, whereas a separate high-frequency channel carries the live gaze stream, a separation we introduce here and justify against the responsiveness budget D2 in section 5.4. Session state is written to a local, file-backed store rather than an external database, a choice we introduce here and defend on grounds of reproducibility and scope in sections 5.5 and 5.7. The external decision provider attaches through a single, well-defined seam rather than being embedded in the host, which is the structural expression of the out-of-scope constraint C3.

The internal structure of the backend host follows one organising principle, namely that dependencies point inward, from volatile detail towards stable abstraction. Hardware access, transport, persistence, and presentation are treated as replaceable detail that depends on the core through abstractions, while the core itself depends on nothing outward [47, 48]. Expressed in the vocabulary of ports and adapters, each external concern meets the core at a port, an interface owned by the core, and each concrete technology is an adapter that the infrastructure supplies behind that port [49]. This is one idea seen from two angles, the dependency rule describing the direction and the ports-and-adapters metaphor describing the seam, and we use both terms in the sections that follow.

This style is not adopted for its own sake, but because it is the most direct structural response to the dominant driver, modular separability D1. Making every boundary an interface, and forbidding the core from depending on any concrete technology, is precisely what allows sensing, analysis, decision, and intervention to be replaced independently, which is the claim that RQ1 puts to the test. The alternative is the shape taken by the earlier Reading the Reader prototypes, in which sensing, adaptation, and presentation

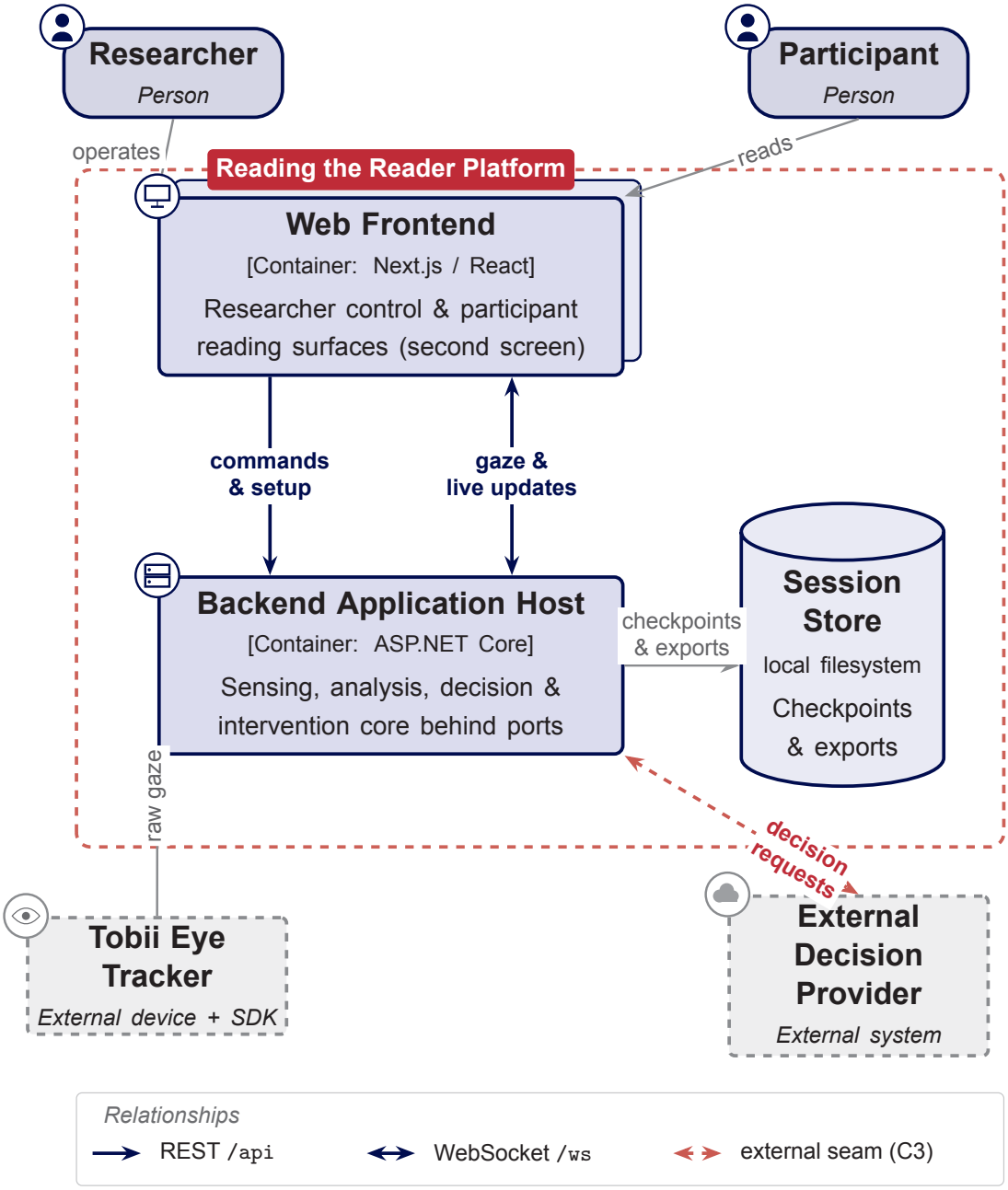


Figure 5.2: Runtime container view of the platform, showing the two front-end surfaces, the backend host, the two communication channels (REST for operator commands and a WebSocket for the live gaze stream), the local session store, and the dashed external-provider seam. Distinct shapes mark the kind of each element (actors, containers, the file-backed store, and external systems), and the legend decodes the relationship styles.

were realised as a single tightly coupled application [5, 6, 7]; that coupling is the limitation identified in section 2.4 and the reason a new architecture is warranted. We defer the full comparison of this and the other structural options to the decision register of section 5.7, and turn first to the concerns upon which the inward-pointing dependencies converge.

5.3 The Four-Module Decomposition

The architectural style of section 5.2 fixes the direction of dependencies but not the concerns they organise. This section opens the core and identifies those concerns. We decompose the platform's adaptive behaviour into four parts (sensing, eye-movement analysis, decision, and intervention), chosen not by technical tier but along the axis of replaceability that the dominant driver D1 demands. Each part is a module that hides its internals behind a contract, and the four contracts together are the structural claim that RQ1 puts to the test.

The four concerns form a pipeline that follows the data, from raw signal to committed change. Sensing is the source: it produces a continuous stream of gaze samples, and its contract is defined over those samples rather than over any device, so that the Tobii tracker and a simulated, mouse-cursor-driven source are interchangeable behind it (D5). Analysis consumes that stream and projects it into oculomotor events (the fixations, saccades, and regressions that constitute reading behaviour), abstracting the particular identification algorithm behind its contract [15]. Decision consumes the analysed state together with the session context and returns, at most, a proposal to intervene; this is the seam at which an external, AI-driven provider stands in for the built-in rule-based strategy, and it is the structural origin of the out-of-scope constraint C3. Intervention is the sink: it validates a proposal and then commits a context-preserving change to the presented text at a controlled boundary, which is the property that RQ3 and driver D7 put to the test. Figure 5.3 shows the four as ports on a common core, a structure we develop in the remainder of this section.

The three concerns that carry intelligence (analysis, decision, and intervention) deliberately share one contract shape. Each is identified by a stable provider identity and exposes a single operation that receives an immutable snapshot of the relevant session state and returns an optional result. The snapshot is the load-bearing choice: a module is handed a copy of what it needs to see, not a reference to the live session, so that it can neither reach into nor mutate the core's state. This is what makes a module replaceable without coordination and testable in isolation, since its entire interaction with the system is one transformation from a snapshot to a result. The uniformity across the three seams is itself a design decision rather than a coincidence, because a contributor who has implemented one kind of module already understands the shape of the others.

These contracts are owned by the core, and that ownership is what gives the dependency rule its force. The interfaces belong to the application core, whereas the concrete implementations (the Tobii adapter, the rule-based and external strategies, and the individual intervention modules) live outside it and depend inward upon them [48]. The core in turn depends on none of these implementations, being expressed entirely in terms of its own ports and domain types. Read concentrically, as in section 5.2, this places the domain and application at the centre and every replaceable detail at the rim, with all dependencies pointing inward. "Independently replaceable" can now be stated precisely: any implementation may be exchanged for another that satisfies the same port without a single change to the core, which is exactly the property evaluated against RQ1.

Replaceability becomes extensibility through one further element at each seam, a registry

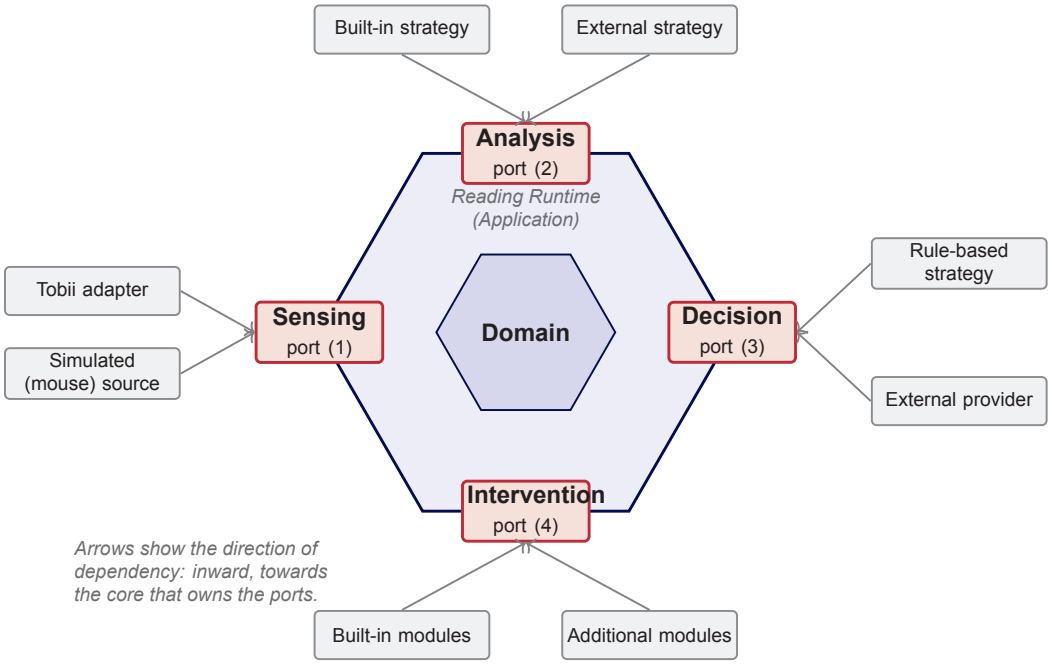


Figure 5.3: The four-module decomposition. Each concern (sensing, analysis, decision, intervention) is a port owned by the application core, while concrete implementations sit outside the core and depend inward upon the ports (solid arrows), so that the core depends on none of them. The numbers indicate the data-flow order of one adaptive cycle, from sensing (1) to intervention (4). Each port resolves its implementation through a registry, so that adding an implementation is an additive change.

that resolves an implementation by its provider identity. Because resolution is by identity rather than by type, adding a new analysis algorithm, decision provider, or intervention is an additive change: a new implementation of an existing port, registered alongside the others, with no edit to the reading runtime that drives them. The platform ships a built-in implementation at each seam and, for analysis and decision, a second implementation that forwards the same contract to an out-of-process provider, so that the extension path is not hypothetical but already exercised by the system itself. This mechanism, a strategy selected at runtime from a registry, is how the modularity named in the title of this thesis is delivered [50].

One consequence of the same discipline deserves emphasis, because it is what makes the architecture practical to develop and to defend. Since the sensing port is written in terms of gaze samples and not Tobii types, the analysis, decision, and intervention layers above it run unchanged when the source is the simulated, mouse-cursor stream rather than the Tobii device. The platform therefore does not require the eye-tracking hardware in order to exercise the adaptive pipeline from end to end, which supports development, automated testing, and demonstration away from the device (D5), and which allows each layer to be validated in isolation.

The decomposition described here is static: it names the four concerns, their contracts, and the direction of their dependencies. What it does not yet show is how the four behave together under the timing constraints of a live session. Section 5.4 sets these ports in motion as the real-time adaptive loop.

5.4 The Real-Time Adaptive Loop

Section 5.3 described the four concerns as a static structure of ports and dependencies. This section sets that structure in motion, as the loop that runs throughout a live reading session, and confronts the concessions that liveness imposes on the clean separation just established. The loop is where the responsiveness driver D2 and the robustness driver D3 are met, and where they stand in their sharpest tension with the modularity of D1.

One adaptive cycle proceeds in the order of the data, and fig. 5.4 traces it across the participants involved. A gaze sample originates at the sensing source and reaches the core, which mirrors it on the dedicated real-time channel to the reading surface and to the researcher's live view. The reading surface, which alone knows the rendered layout, maps the sample to a position in the text and returns an enriched observation to the core. The core's analysis layer projects the stream of observations into oculomotor events, and a change in that analysed state triggers the decision layer to evaluate, drawing on either the built-in strategy or an external provider. The decision yields at most one intervention proposal, which is either applied at once or, in advisory operation, surfaced to the researcher and applied only on approval. Applying it produces an adapted view and an intervention event, which the core returns to the reading surface to re-render, closing the cycle.

The cycle just described conceals a deliberate split in timescale that is the key to meeting the responsiveness budget D2. High-frequency gaze does not travel the request and response path used for commands and setup; it flows on a separate real-time channel, so that mirroring a sample to the live views never waits on the slower control path. On that hot path the core does the minimum: it sanitises the sample, retains it as the latest value, and broadcasts it, without acquiring the lock that guards session lifecycle and decision state. The heavier modular pipeline of analysis, decision, and intervention runs at the cadence of analysed-state changes rather than once per raw sample, so the indirection introduced for modularity in section 5.3 is kept off the per-sample path. The end-to-end latency that this arrangement must respect, from gaze to committed intervention, is the budget evaluated in chapter 7 under RQ2.

This split is the architecture's answer to a tension we do not wish to hide. The cleanest per-concern indirection is rarely the fastest at runtime, and an intervention delivered after the moment at which it remains valid is of no use to the reader. We reconcile separation with speed not by diluting the contracts of section 5.3 but by confining their indirection to the lower-frequency control path, leaving the sensing path thin and direct.

Where an intervention is committed matters as much as when, because a typographic change alters the very layout the participant is reading. The intervention contract is therefore two-phase: a proposal is first validated against the current reading context and only then committed, and the commit is performed at a controlled boundary rather than at an arbitrary instant (D7). Each commit carries an anchor, a sentence, token, or block together with a measured anchor error and the viewport displacement, so that the reading surface can restore the reader's position across the change. This commit discipline is the architectural expression of the recovery concern reported in the literature [10]: it is what allows a micro-intervention to adapt the text without costing the participant their place, which is the property RQ3 examines.

The loop also keeps the researcher in control of how decisions are enacted. It supports an autonomous mode, in which a proposal permitted by the active policy is applied immediately, and an advisory mode, in which the same proposal is presented for the researcher

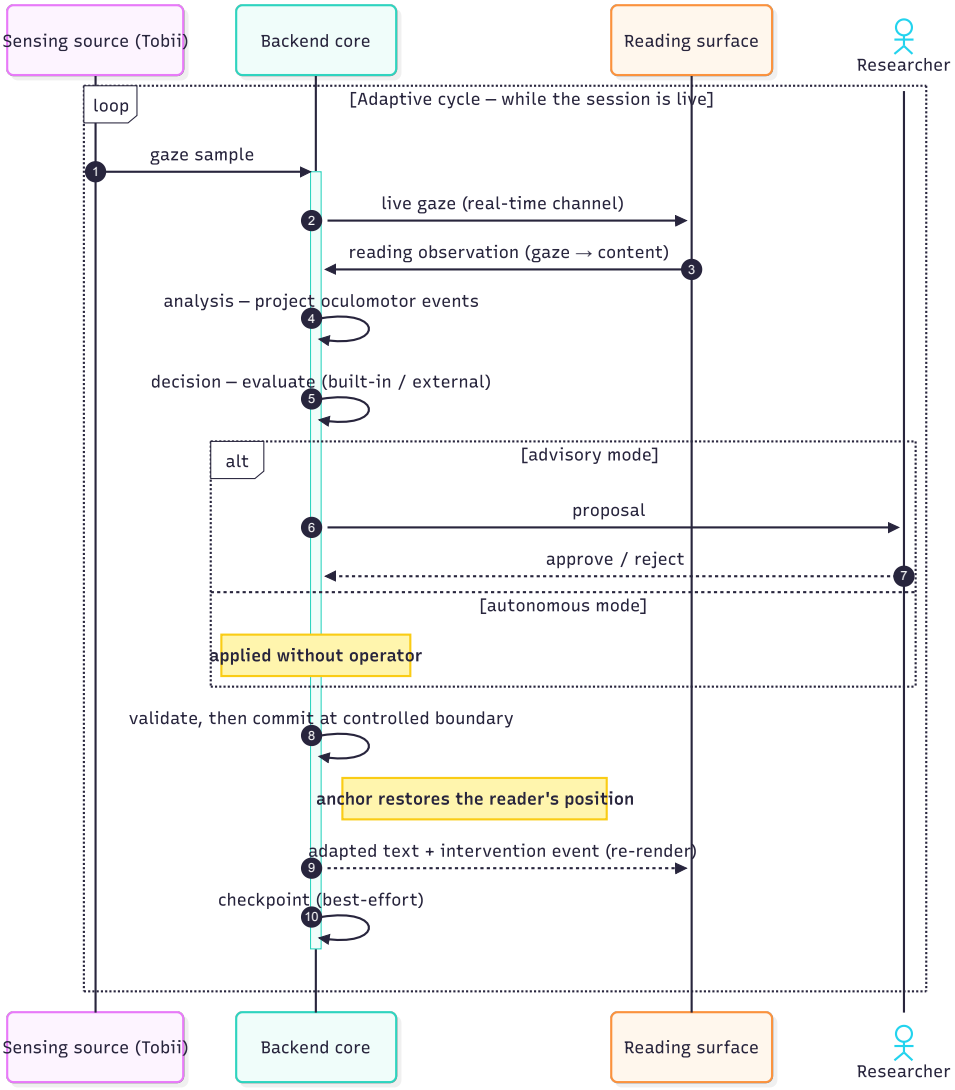


Figure 5.4: One iteration of the real-time adaptive loop. High-frequency gaze is mirrored to the reading surface on the dedicated real-time channel; the surface returns a content-anchored observation; the core projects oculomotor events, evaluates a decision (using the built-in strategy or an external provider) and, in advisory mode, awaits researcher approval before validating and committing the intervention at a controlled boundary; the adapted text is then returned to the surface to re-render. The self-calls on the backend core denote the analysis, decision, intervention, and persistence steps performed within it, and the checkpoint is written on a best-effort basis.

to approve or reject before it is committed. The control surface through which this happens is the subject of section 5.6; here it is enough to note that the loop accommodates both an unattended and a supervised experiment without structural change.

Because a reading session involves an invited participant and cannot be repeated cheaply, the loop is built to survive disturbance rather than to assume its absence. It degrades gracefully: a client that disconnects falls out of the broadcast without interrupting the stream, and a decision provider that returns nothing, or is not present, yields no proposal and the cycle continues unaffected. It also checkpoints its state, so that an interrupted session is recoverable: every state-changing step writes a checkpoint while holding the session lock, and a background worker independently persists the session at a fixed interval and again at shutdown. This checkpointing is best-effort by design, swallowing its own failures, so that the durability mechanism can never itself halt a running session (D3).

The loop continually emits events and state changes, and those that are checkpointed accumulate into a record of the session. That record is not merely a recovery aid but the platform's primary research output, and section 5.5 turns from the live loop to the authoritative account it leaves behind.

5.5 Session Record and Reproducibility

Section 5.4 closed with the loop emitting a continuous stream of events and state changes. This section turns to the artifact into which those events accumulate, the authoritative session record, which is the platform's primary research output rather than an incidental by-product. It is the structure through which the reproducibility driver D4 is met and the criterion against which RQ4 is judged.

Reproducibility (D4) is met by a single record, owned by the backend, that is designed so a completed session can be reconstructed from the record alone. This is the need expressed by the Data Consumer stakeholder (section 4.1), who must be able to analyse a session without access to the running system that produced it. To make that possible the record is self-describing: it is identified by a named schema and an integer version, and it opens with a manifest recording its provenance, the producing application, the backend, and the exporter that wrote it. A consumer therefore knows exactly which shape of record it holds and what produced it, and the platform can evolve the record across versions without silently invalidating earlier exports.

What the record contains is everything required to reconstruct a run, organised into labelled sections rather than a flat list. It captures the experiment context and the active condition (the parameters of the run), the reading content together with a content hash and the identity of the tokenization that segmented it, the calibration summary, the screen geometry on which the text was presented, and the presentation baseline (the typographic configuration before any intervention). To these it adds the time series the session generated: the raw sensing stream, the derived oculomotor and attention events, the decision proposals and the interventions they produced, the participant's comprehension responses, and any annotations the researcher attached. Every one of these entries is stamped with a monotonic sequence number and a timestamp, so the record is a deterministically ordered log rather than an unordered dump, and it is this ordering that makes a faithful reconstruction possible.

A feature of this record that is easy to miss is that it is not assembled after the session ends. It is the same structure that the checkpoint worker of section 5.4 writes to disk continuously while the session runs (fig. 5.5), so the durability checkpoint and the research export are one artifact serving two roles rather than two parallel representations that might

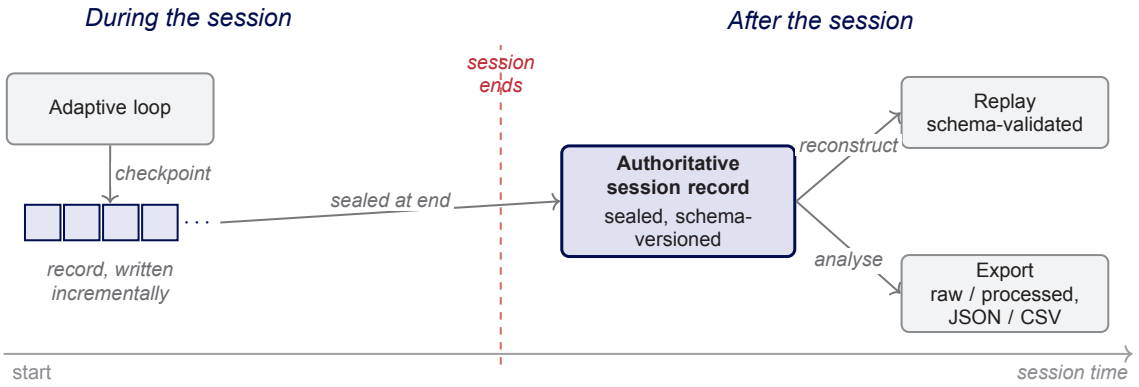


Figure 5.5: The lifecycle of the session record. During a session the adaptive loop writes the record incrementally through periodic checkpoints; at session end the same artifact is sealed as the authoritative, schema-versioned record; afterwards it is reconstructed by schema-validated replay and exported in a raw or processed profile as JSON or CSV. The record written live during the session and the record later replayed or exported are one and the same artifact.

disagree. One consequence is that an interrupted session and a completed session yield the same kind of record, which is why recovery (D3) reduces to resuming from the most recent materialization of the record. The record is thus authoritative in a strong sense: there is no second account of the session against which it could be inconsistent.

Whether the record truly captures enough is a claim we prefer to demonstrate rather than assert. The platform reconstructs a session from a record by replaying it, and the importing side validates the record against the schema before reconstruction begins, so a record that is incomplete or malformed is rejected rather than silently mis-replayed. Replay is therefore the operational definition of completeness: if a session can be re-presented and re-analysed from the record alone, the record contained what reproducibility requires, and if it cannot, the record is by definition deficient. This is precisely the property that RQ4 examines, and it is why replay (FR21) is treated as a first-class capability of the platform rather than a convenience.

Because different analyses want the data in different shapes, the same underlying record is offered in more than one profile and serialisation. A full, raw export preserves every captured sample for re-analysis, whereas a processed export carries the derived events better suited to immediate statistical work, and either may be written as structured JSON or as tabular CSV for tools that expect rows. The schema doubles as the boundary of disclosure: because gaze and comprehension responses are personal data under the GDPR (C4), fixing in the schema exactly what the record captures and exports makes the scope of personal data explicit and bounded. Participant consent for that capture remains a procedural matter handled around the session rather than a property of the architecture, consistent with the scope set in section 5.1.

With the session record, the account of the running platform is complete: it can sense, analyse, decide, intervene, and leave behind a faithful and reproducible record of having done so. What remains is to show how the architecture is opened for others to extend, and section 5.6 turns from the platform's behaviour to the seams through which new interventions and external decision providers are added.

5.6 Extensibility and the External-Provider Seam

Section 5.3 declared the seams between the four concerns, and sections 5.4 and 5.5 showed the platform exercising its own built-in implementations behind them. This section shows those seams used by others. Two paths extend the platform, and the architecture is arranged so that both are additive: neither requires a change to the reading runtime or to the existing modules.

The two paths differ in where the new behaviour runs. A new implementation may be added in-process, behind an existing port, so that it executes inside the platform alongside the built-in modules. Alternatively, an entire concern may be delegated out-of-process to an external provider that runs as a separate program and communicates with the platform across a uniform seam. The first path is how the platform's own catalogue of interventions and strategies grows; the second is how the decision intelligence that this thesis holds out of scope (C3) can later be supplied by another team.

Adding a new intervention is the smallest unit of extension and illustrates the in-process path in full. A contributor implements the intervention contract of section 5.3, providing the descriptor that identifies the module, the validation that checks a request against the current context, and the execution that commits the change, and then registers the implementation with the container. From that point the registry the runtime consults discovers the module by its identity, and the module becomes available to the decision layer and selectable by the operator. No part of the reading runtime, the adaptive loop, or any other module is edited to accommodate it, so the intervention is present solely because it was registered, which is precisely the additive extension that RQ1 requires.

The out-of-process path is the architectural answer to constraint C3, which places end-to-end decision intelligence outside the scope of this thesis. Rather than embed such intelligence, the platform exposes a uniform module-provider seam through which an external program can supply a concern, so that the thesis's responsibility is to make that seam well-defined rather than to implement what crosses it. A concern that may be externalised contributes two small pieces: a module definition that names the seam and the messages that cross it, and an inbound handler that interprets what a provider sends back. An external provider, running as a separate process, connects over a provider transport and registers itself as the active provider for that module's identity.

With a provider attached, the seam routes a concern out and back, as fig. 5.6 traces. When the core reaches the seam it asks the gateway whether a provider is active for the module in question; if one is, it publishes the current context to that provider and later ingests, through the inbound handler, the proposal the provider returns. If no provider is active, the gateway reports as much and the same concern is served by its in-process built-in implementation, so the two paths meet at a single outcome. Because the seam is keyed by module identity and falls back to the built-in whenever no provider is attached, an external provider is a genuine plug-in: the loop of section 5.4 runs unchanged whether one is present or absent, which is the graceful behaviour required of live operation (D3).

Extensibility opens the platform to future module providers; a second structure, the operator control plane, opens it to the researcher who runs the experiment, and it too is an architectural concern rather than a matter of screen layout. The architecture separates the researcher's control surface from the participant's reading surface, the second-screen arrangement introduced in section 5.2, so that the participant sees only the text while the researcher retains the controls and a mirror of the participant's view. It gates a session behind explicit readiness, requiring device selection, calibration, and validation to succeed before a run can begin, so that a session cannot be started in a mis-configured

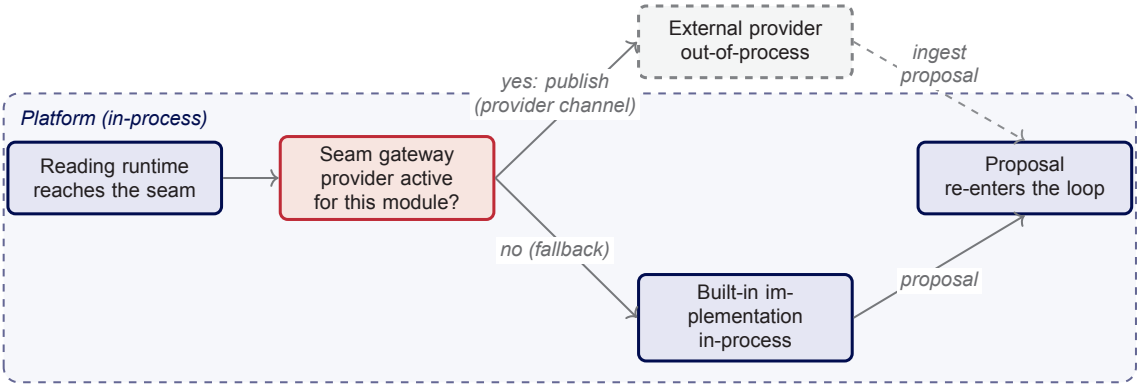


Figure 5.6: The external-provider seam and its fallback. When the core reaches a seam it asks the gateway whether an external provider is active for that module; if one is, it publishes the context across the provider channel and ingests the proposal the out-of-process provider returns, and if not the concern is served by its in-process built-in implementation. Either way a single proposal re-enters the loop, so an external provider is a plug-in that the platform runs with or without.

state. And it derives the researcher's view from the single authoritative session snapshot of section 5.5, so that the operator commands and observes the same state through one consistent plane rather than a collection of disconnected controls.

Through that same plane the researcher selects the autonomous or advisory mode of section 5.4, deciding for each experiment how much of the adaptive loop to retain under human approval, which is the operator control and experimentability that RQ4 examines. With the extension seams and the control plane, the structures that make up the architecture are now all in view. What remains is to draw together the decisions that produced them, each with the alternatives it was chosen over, and section 5.7 gathers these into a single register.

5.7 Design Decision Register

The decisions that produced the structures of this chapter were argued in place, each against the driver it serves. This section consolidates them into a single register, so that each can be inspected together with the alternative it was chosen over and the reason for the choice. Table 5.2 is the chapter's defence in compact form, and its rows are the design commitments that chapter 7 tests against the research questions.

Table 5.2: Consolidated design decision register. Each decision is shown with the architectural driver(s) it serves (Dn as in table 5.1), the principal alternative considered, and the rationale for the choice. The decisions are argued in place in sections 5.2 to 5.6 and are the design commitments evaluated in chapter 7.

ID	Decision	Drivers	Alternative	Rationale
DR1	Inward-dependency layering (ports and adapters)	D1	A single coupled application, the shape of the prior Reading the Reader prototypes	Only an interface seam per concern with inward dependencies makes sensing, analysis, decision, and intervention independently replaceable; the coupling of the prototypes is the documented gap (RQ1).
DR2	Uniform seam shape with a per-concern registry	D1	Bespoke per-concern interfaces; direct handles to live session state	A uniform shape (an identity plus a snapshot in and an optional result out) makes seams predictable and modules additive, and immutable snapshots prevent a module from mutating core state, keeping it replaceable and testable in isolation.
DR3	Dedicated real-time channel for gaze, separate from the request/response path	D2	Carrying gaze over the request/response path or through application state	The per-sample path must stay lock-free and must not wait on the control path; routing high-frequency gaze separately keeps the end-to-end loop latency within budget (RQ2).
DR4	External concerns through a uniform, identity-keyed provider seam	D1, C3	Embedding decision or analysis intelligence in the core	The constraint C3 places end-to-end intelligence out of scope; a seam keyed by module identity, with fallback to the built-in implementation, makes integration well-defined without supplying the algorithms (RQ1).
DR5	Two-phase validate-then-commit intervention at a controlled boundary	D7	Applying an intervention immediately at an arbitrary instant	Committing at a controlled boundary and carrying an anchor lets the text adapt while the reader's position is preserved, so adaptation does not cost the participant their place (RQ3).
DR6	One authoritative, schema-versioned session record	D4	Client-held state, or several parallel representations of the session	A single backend-owned record, reconstructable by replay, guarantees there is no second account against which it could disagree, and an explicit schema version lets the record evolve without invalidating earlier exports (RQ4).

continued on next page

table 5.2 continued from previous page

ID	Decision	Drivers	Alternative	Rationale
DR7	File-backed, checkpointed local store	D3, D4	An external database	The single-operator scope and file-only reading material do not warrant a database; periodic, best-effort checkpointing of the record provides recovery without adding operational surface.
DR8	Sensing-source port behind an adapter	D5, C1	Calling the Tobii SDK directly from the core	Quarantining the Windows-only SDK (C1) behind a port admits a simulated source alongside the device, enabling development, testing, and demonstration without hardware and exercising each layer in isolation.
DR9	Autonomous and advisory execution modes	D6	An autonomous-only loop with no operator gate	A researcher-operated instrument needs the option of human approval per experiment; supporting both modes without structural change is what gives the researcher control over experimentability (RQ4).

Read together, the register makes the chapter's recurring tension explicit. Several decisions (DR3, DR7, and the fallback within DR4) trade a measure of architectural purity for responsiveness or robustness, while others (DR1, DR2, DR8) hold the separation of concerns firm; none of these is an accident of implementation, and each is a deliberate position taken in view of the drivers. That the trade-offs are visible and motivated, rather than hidden, is what we intend by presenting the architecture as argued rather than narrated.

5.8 Summary

This chapter developed and justified the architecture of the Reading the Reader platform, the central contribution of the thesis. It first isolated the architecturally significant requirements as seven design drivers and adopted an inward-dependency, ports-and-adapters style as the structural response to the dominant driver of modular separability. On that foundation it decomposed the platform's adaptive behaviour into four independently replaceable concerns (sensing, eye-movement analysis, decision, and intervention), each hidden behind a uniform contract and resolved through a registry, which is the modularity that RQ1 requires. It set these concerns in motion as a real-time adaptive loop that reconciles modular separation with the responsiveness and robustness drivers, by confining the modular indirection to a lower-frequency control path while keeping the per-sample sensing path thin, and that commits interventions at controlled boundaries so that adaptation preserves the reader's position, as RQ2 and RQ3 require. It described the single authoritative, schema-versioned session record that makes a session reproducible from the record alone and demonstrable through replay, the experimentability and auditable record that RQ4 examines. Finally, it showed how the architecture is opened for extension, in-process through additive registration and out-of-process through a uniform provider seam, and gathered the design decisions and their alternatives into a single register. The architecture established here is the specification that chapter 6 realises in code,

where selected implementation highlights, the build, and the development workflow are described.

6 Implementation

6.1 Code Structure and Solution Layout

Sections 5.2 and 5.3 described the architecture as concerns, ports, and a rule that dependencies point inward. This chapter goes inside that description to show how it is realised in code, beginning here with where the code lives before section 6.2 follows a session through it. The system is a monorepo with two independent roots, a .NET backend solution under `Backend/` and a Next.js front-end under `Frontend/`, with no shared workspace manifest binding them, so that each is built and versioned on its own. Their folder layout is shown in fig. 6.1 and their dependency structure in fig. 6.2.

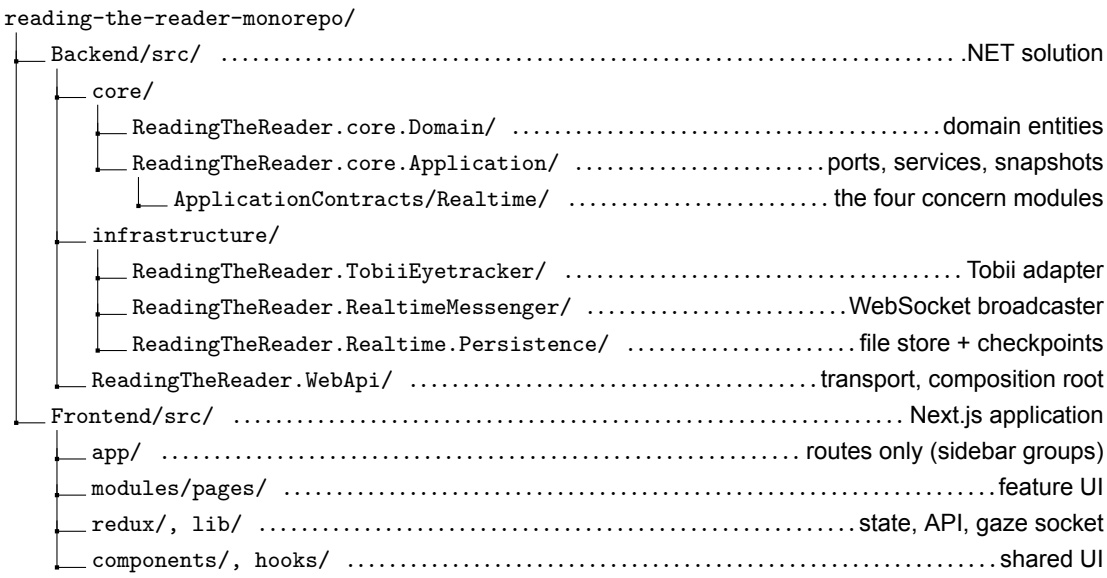
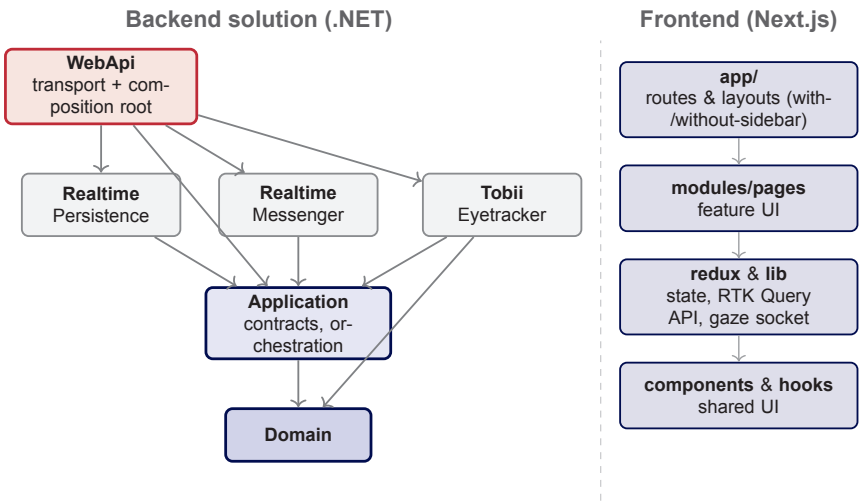


Figure 6.1: Curated layout of the monorepo (selected folders only). The backend groups projects into `core` (the `Domain` entities and the `Application` with its ports), `infrastructure` (the adapters), and the `WebApi` composition root; the four concern modules of section 5.3 appear as folders beneath the application contracts. The front-end separates routing (`app/`) from feature UI, shared state and transport, and shared components.

The backend solution realises the concentric structure of section 5.2 directly as projects. A `Domain` project at the centre holds the basic entities, an `Application` project holds the use-case services, the snapshots, and the port interfaces, three infrastructure projects supply the adapters for the Tobii device, realtime messaging, and persistence, and a `WebApi` project provides the HTTP and WebSocket transport and composes the whole at startup. Each conceptual layer of the architecture is thus a separate compilation unit, which is what allows the dependency rule to be enforced rather than merely intended.

That rule is visible in the project references, and because a project cannot compile against a reference it does not declare, the references are evidence rather than documentation. The `Domain` project references nothing; the `Application` project references only `Domain`; and each infrastructure adapter references inward to `Application`, with the Tobii project additionally referencing `Domain`, so that every reference points towards the centre, as fig. 6.2 shows. The one project that references outward is `WebApi`, which depends on



*Arrow $A \to B$: project (or layer) A references / depends on B. In the backend every reference points inward, towards the **Domain** core.*

Figure 6.2: Code structure of the two monorepo roots. In the backend solution every project reference points inward (`WebApi`, the composition root, depends on `Application` and all infrastructure; each infrastructure adapter and `Application` depend inward towards `Domain`), which is the dependency rule of section 5.2 enforced by the build rather than merely intended. The front-end is a single Next.js application whose source separates thin routing (`app/`) from feature UI (`modules/pages`), shared state and transport (`redux`, `lib`), and shared components.

`Application` and on all three infrastructure projects at once. This is deliberate, and it is the single exception the style permits: as the composition root, `WebApi` alone must know the concrete adapters in order to register them against the core's ports at startup, while the core itself never names them.

The front-end is a single Next.js application whose source mirrors the same discipline of separating routing from logic. The `app/` directory contains only routes and layouts, organised into two route groups, a researcher-facing group with the operator sidebar and a participant-facing group without it, which are the two surfaces of the container view of section 5.2. Feature UI lives in `modules/pages`, cross-feature state and the typed REST clients in `redux`, and shared helpers including the realtime gaze socket in `lib`, with common components and hooks factored into `components` and `hooks`. Routing therefore stays thin and hands off to feature logic that is kept separately, in the same spirit as the backend's separation of transport from core.

Because there is no generated package shared between the two roots, the TypeScript types the front-end uses are hand-written mirrors of the backend records, and keeping the two aligned is a manual discipline rather than a compiler-checked one. With the territory mapped, the next section puts it in motion: section 6.2 follows a single experiment session through these projects, from the researcher's first setup action to the sealed record left behind.

6.2 Anatomy of an Experiment Session

§6.2 to be written. Trace one real experiment session end-to-end through the code: setup -> device/calibration (readiness gating) -> reading with the live adaptive loop -> comprehension/quiz -> export/seal. ExperimentSessionManager as orchestrator, session lifecycle/state, authoritative snapshot. Session state diagram + UI screenshots in context (ask user). Realizes RQ1, RQ3, RQ4.

6.3 Implementation Highlights

§6.3 to be written. 2–4 deeper dives the walkthrough flags: real-time gaze hot path (lock-free, WebSocket; RQ2); pluggable modules + a worked "add an intervention" (RQ1); external-provider framework realized; session record schema + replay (RQ4). Each: prose + at most one short listing. Pick the dives during scaffolding.

6.4 User Interface Realization

§6.4 to be written. Researcher control surface and participant reader as two faces of one front-end; calibration, live monitor / second-screen mirror, replay viewer. Brief rationale only for decisions that matter (readiness gating UI); cref Ch5 sec:design-extensibility rather than re-arguing. Representative screenshots in body; fuller set -> UI gallery appendix. Ask user for screenshots.

6.5 Engineering Practice

§6.5 to be written. Build & CI (frontend-ci, backend-ci; per-subproject deps; Windows/Tobii constraint); cross-cutting concerns (error handling conventions, hot-path vs gated concurrency, configuration, logging style); testing approach; challenges & resolutions (gaze-to-content mapping over Markdown, latency, Windows-only Tobii SDK, context preservation across re-render).

6.6 Summary

§6.6 to be written last. Recap what was realized (code structure, the session in motion, selected realizations, the UI, engineering practice) and hand off to chapter 7, which measures how well the realized system meets RQ1–RQ4.

7 Evaluation

8 Discussion

8.1 Achievements vs. Problem Statement
8.2 Comparison with Related Work
8.3 Limitations
8.4 Lessons Learned
8.5 Future Work

9 Conclusion

Bibliography

[1] World Health Organization. *Blindness and Vision Impairment*. 2026. URL: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment (visited on 06/08/2026).

[2] Rajendra S. Apte. "Age-Related Macular Degeneration". In: *New England Journal of Medicine* 385.6 (2021), pp. 539–547. DOI: 10.1056/NEJMcp2102061.

[3] DTU Compute. *Reading the Reader*. DTU Compute Research. 2025. URL: https://people.compute.dtu.dk/pgba/research/reading-the-reader/ (visited on 01/01/2025).

[4] Novo Nordisk Foundation. *Artificial Intelligence Will Help to Improve Reading Opportunities for Millions of People*. 2023. URL: https://novonordiskfonden.dk/en/news/artificial-intelligence-will-help-to-improve-reading-opportunities-for-millions-of-people/ (visited on 06/08/2026).

[5] Lars Kristian Burchard Refsgaard and Aamir Ahmad Farooq. "Reading the Reader: An Intelligent System for Optimizing Typography for Central Vision Loss Readers". Supervised by Per Bækgaard and Chaudhary Muhammad Aqdus Ilyas. Master's thesis. Technical University of Denmark, DTU Compute, 2024. URL: https://findit.dtu.dk/en/catalog/65fa375f2a83a81e2e536b41 (visited on 06/03/2026).

[6] Luz Emma Pereira and Mihai-Valentin Andrei. "Designing a Typography-Adaptive Reading Interface for Individuals with Central Vision Loss". Supervised by Per Bækgaard and Chaudhary Muhammad Aqdus Ilyas. Master's thesis. Technical University of Denmark, DTU Compute, 2024. URL: https://findit.dtu.dk/en/catalog/669da34561fca2e8f32bfbd8 (visited on 06/03/2026).

[7] Marko Palinec. "Reactive Reading: Crafting an Adaptive Frontend Interface for Dynamic Machine Learning". Supervised by Per Bækgaard. Master's thesis. Technical University of Denmark, DTU Compute, 2024. URL: https://findit.dtu.dk/en/catalog/66bbf5cfdeca9be8f357d145 (visited on 06/03/2026).

[8] Alan R. Hevner et al. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: 10.2307/25148625.

[9] Dan Roland Persson et al. "A Design Framework for Microintervention Software Technology in Digital Health: Critical Interpretive Synthesis". In: *Journal of Medical Internet Research* 27 (2025), e72658. DOI: 10.2196/72658.

[10] Helena Eschricht Jensen et al. "Context Preservation Through Eye Tracking: Adaptive Reading Application Design for an Optimal Reading Experience". In: *Proceedings of the 2025 Symposium on Eye Tracking Research and Applications (ETRA)*. 2025. DOI: 10.1145/3715669.3725897. URL: https://orbit.dtu.dk/files/406853513/3715669.3725897.pdf.

[11] E. Kanonidou. "Reading performance and central field loss". In: *Hippokratia* 15.2 (2011), pp. 103–108.

[12] Susana T. L. Chung. "Improving Reading Speed for People with Central Vision Loss through Perceptual Learning". In: *Investigative Ophthalmology & Visual Science* 52.2 (2011), pp. 1164–1170. DOI: 10.1167/iovs.10-6034. URL: https://doi.org/10.1167/iovs.10-6034.

[13] M. B. Coco-Martin et al. "Training Reading Skills in Central Field Loss Patients: Impact of Clinical Advances and New Technologies to Improve Reading Ability". In: *Visual Impairment and Blindness*. 2020. DOI: 10.5772/intechopen.88943. URL: https://doi.org/10.5772/intechopen.88943.

[14] Lilit Hakobyan et al. “Mobile assistive technologies for the visually impaired”. In: *Survey of Ophthalmology* 58.6 (2013), pp. 513–528. DOI: 10.1016/j.survophthal.2012.10.004. URL: https://doi.org/10.1016/j.survophthal.2012.10.004.

[15] Dario D. Salvucci and Joseph H. Goldberg. “Identifying fixations and saccades in eye-tracking protocols”. In: *Proceedings of the 2000 Symposium on Eye Tracking Research & Applications (ETRA ’00)*. 2000, pp. 71–78. DOI: 10.1145/355017.355028. URL: https://doi.org/10.1145/355017.355028.

[16] Xiu Guan, Chao-Jing Lei, and Xiang Feng. “The Predictive Effect of Eye-Movement Behavior on Reading Performance in Online Reading Contexts”. In: *Proceedings of the 14th International Conference on Education Technology and Computers (ICETC ’22)*. 2022, pp. 470–475. DOI: 10.1145/3572549.3572624. URL: https://doi.org/10.1145/3572549.3572624.

[17] Aries Arditi. “Adjustable typography: An approach to enhancing low vision text accessibility”. In: *Ergonomics* 47.5 (2004), pp. 469–482. DOI: 10.1080/0014013031000085680. URL: https://doi.org/10.1080/0014013031000085680.

[18] Sofie Beier et al. “Increased letter spacing and greater letter width improve reading acuity in low vision readers”. In: *Information Design Journal* 26.1 (2021), pp. 73–88. DOI: 10.1075/idj.19033.bei. URL: https://doi.org/10.1075/idj.19033.bei.

[19] Luz Rello, Martin Pielot, and Mari-Carmen Marcos. “Make It Big! The Effect of Font Size and Line Spacing on Online Readability”. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI ’16)*. 2016, pp. 3637–3648. DOI: 10.1145/2858036.2858204. URL: https://doi.org/10.1145/2858036.2858204.

[20] Luz Rello and Jeffrey P. Bigham. “Good Background Colors for Readers: A Study of People with and without Dyslexia”. In: *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS ’17)*. 2017, pp. 72–80. DOI: 10.1145/3132525.3132546. URL: https://doi.org/10.1145/3132525.3132546.

[21] David Beymer, Daniel M. Russell, and Peter Orton. “An eye tracking study of how font size and type influence online reading”. In: *Proceedings of the 22nd British HCI Group Annual Conference (People and Computers XXII)*. 2008. DOI: 10.1145/1531826.1531831. URL: https://doi.org/10.1145/1531826.1531831.

[22] R. Biedert, Georg Buscher, and A. Dengel. “The eyeBook: Using eye tracking to enhance the reading experience”. In: *Informatik-Spektrum* (2010), pp. 272–281. DOI: 10.1007/s00287-009-0381-2. URL: https://doi.org/10.1007/s00287-009-0381-2.

[23] Manuel Landsmann, Olivier Augereau, and Koichi Kise. “Classification of Reading and Not Reading Behavior Based on Eye Movement Analysis”. In: *Adjunct Proceedings of the 2019 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp/ISWC ’19 Adjunct)*. 2019, pp. 109–112. DOI: 10.1145/3341162.3343811. URL: https://doi.org/10.1145/3341162.3343811.

[24] Tobii Pro. *Tobii Pro Fusion*. Tobii Pro Product Page. 2025. URL: https://www.tobii.com/products/eye-trackers/screen-based/tobii-pro-fusion (visited on 06/03/2026).

[25] SR Research. *EyeLink 1000 Plus*. SR Research Product Page. 2025. URL: https://www.sr-research.com/eyelink-1000-plus/ (visited on 06/03/2026).

[26] SR Research. *EyeLink 1000 Plus – Request a Quote*. 2025. URL: https://www.sr-research.com/request-a-quote/eyelink-1000-plus-request-a-quote/ (visited on 06/03/2026).

[27] GazePoint. *GazePoint GP3 HD – Shop Page*. GazePoint Official Store. 2025. URL: https://www.gazept.com/shop/ (visited on 06/03/2026).

[28] Pupil Labs. *Neon Eye Tracking Platform*. 2025. URL: https://pupil-labs.com/products/neon (visited on 06/03/2026).
[29] RealEye. *RealEye Pricing – Support Documentation*. 2025. URL: https://support.realeye.io/realeye-pricing (visited on 06/03/2026).
[30] RealEye. *RealEye Pricing*. 2025. URL: https://www.realeye.io/pricing (visited on 06/03/2026).
[31] Tobii Pro. *Tobii Pro Lab*. 2025. URL: https://www.tobii.com/products/software/behavior-research-software/tobii-pro-lab (visited on 06/03/2026).
[32] Lexplore. *Lexplore Assessment*. 2025. URL: https://lexplore.com/lexplore-assessment (visited on 06/03/2026).
[33] Lexplore. *Lexplore Pricing*. 2025. URL: https://lexplore.com/pricing-lexplore (visited on 06/03/2026).
[34] OrCam. *OrCam Read 3*. 2025. URL: https://www.orcam.com/en-us/orcam-read-3-new (visited on 06/03/2026).
[35] Microsoft. *Use Immersive Reader in Word*. 2025. URL: https://support.microsoft.com/en-au/office/use-immersive-reader-in-word-a857949f-c91e-4c97-977c-a4efcaf9b3c1 (visited on 06/03/2026).
[36] Amazon. *Kindle Text Display Settings*. 2025. URL: https://www.amazon.com/gp/help/customer/display.html?nodeId=T5Y94BzSCGwm0vd75W (visited on 06/03/2026).
[37] Ken Peffers et al. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: 10.2753/MIS0742-1222240302.
[38] Design Council. *The Double Diamond*. Design Council. 2023. URL: https://www.designcouncil.org.uk/our-resources/the-double-diamond/ (visited on 06/03/2026).
[39] Franziska Schorr and Lars Hvam. "Design Science Research: A Suitable Approach to Scope and Research IT Service Catalogs". In: *Proceedings of the 2018 IEEE World Congress on Services (SERVICES)*. IEEE, 2018, pp. 25–26. DOI: 10.1109/SERVICES.2018.00026. URL: https://backend.orbit.dtu.dk/ws/portalfiles/portal/184223021/Article_How_to_design_an_IT_service_catalog.pdf (visited on 06/08/2026).
[40] Dai Clegg and Richard Barker. *CASE Method Fast-Track: A RAD Approach*. Addison-Wesley, 1994. ISBN: 9780201624328. URL: https://www.worldcat.org/isbn/9780201624328 (visited on 06/03/2026).
[41] Keith Rayner. "Eye Movements in Reading and Information Processing: 20 Years of Research". In: *Psychological Bulletin* 124.3 (1998), pp. 372–422. DOI: 10.1037/0033-2909.124.3.372.
[42] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88. 2016. URL: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679 (visited on 06/03/2026).
[43] Robert B. Miller. "Response time in man-computer conversational transactions". In: *Proceedings of the December 9–11, 1968, Fall Joint Computer Conference, Part I (AFIPS '68)*. Association for Computing Machinery, 1968, pp. 267–277.
[44] Jakob Nielsen. *Response Times: The 3 Important Limits*. Nielsen Norman Group. 1993. URL: https://www.nngroup.com/articles/response-times-3-important-limits/ (visited on 06/10/2026).
[45] Lianping Chen, Muhammad Ali Babar, and Bashar Nuseibeh. "Characterizing Architecturally Significant Requirements". In: *IEEE Software* 30.2 (2013), pp. 38–45. DOI: 10.1109/MS.2012.174.

[46] Simon Brown. *The C4 Model for Visualising Software Architecture*. 2026. URL: https://c4model.com/ (visited on 06/08/2026).
[47] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson, 2017. ISBN: 9780134494166. URL: https://www.pearson.com/en-us/subject-catalog/p/clean-architecture-a-craftsmans-guide-to-software-structure-and-design/P200000009528 (visited on 06/08/2026).
[48] Robert C. Martin. "The Dependency Inversion Principle". In: *C++ Report* 8.6 (1996), pp. 61–66. ISSN: 1040-6042.
[49] Alistair Cockburn. *Hexagonal Architecture (Ports and Adapters)*. 2005. URL: https://alistair.cockburn.us/hexagonal-architecture/ (visited on 06/08/2026).
[50] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 9780201633610.

A Declaration of Use of Generative AI

A.1 Statement of Authorship and Responsibility

We, the authors of this thesis, take full and sole responsibility for the content of this report. All decisions regarding scope, claims, results, and interpretations are ours. Generative artificial intelligence (GAI) tools listed below were used as assistive instruments only; they are not co-authors of this work.

A.2 Tools Used

Tool	Version / Date	Purpose

Table A.1: Generative AI tools used during the preparation of this thesis.

A.3 Where and How GAI Was Used

A.3.1 Writing Assistance

A.3.2 Code Assistance

A.3.3 Literature Search and Synthesis

A.3.4 Figures, Tables, or Data

A.4 What GAI Was Not Used For

- Formulation of the problem statement and research questions.
- Selection and reading of all cited sources.
- Design decisions for the system architecture.
- Evaluation methodology and interpretation of results.
- Conclusions and discussion of limitations.

A.5 Prompt and Output Retention

Representative prompts and outputs are retained by the authors and can be made available to the supervisor or examiners upon request, in accordance with DTU guidance on documenting GAI use.

A.6 Citations of GAI Output

Where text, code, or figures derived from GAI output appear in the body of the thesis, they are cited inline using the format below.

> Anthropic. *Claude* (Opus 4.7) [Large Language Model]. https://claude.ai. Accessed YYYY-MM-DD.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Bygning 324,
2800 Kgs. Lyngby
Tlf. 4525 1700

https://www.compute.dtu.dk/